

Προηγμένα Θέματα Αρχιτεκτονικής Υπολογιστών

Υποθετική Εκτέλεση Εντολών (Hardware-Based Speculation)

Νεκτάριος Κοζύρης, Διονύσης Πνευματικάτος &
Κωστής Νίκας

{nkoziris,pnevmati,knikas}@cslab.ece.ntua.gr

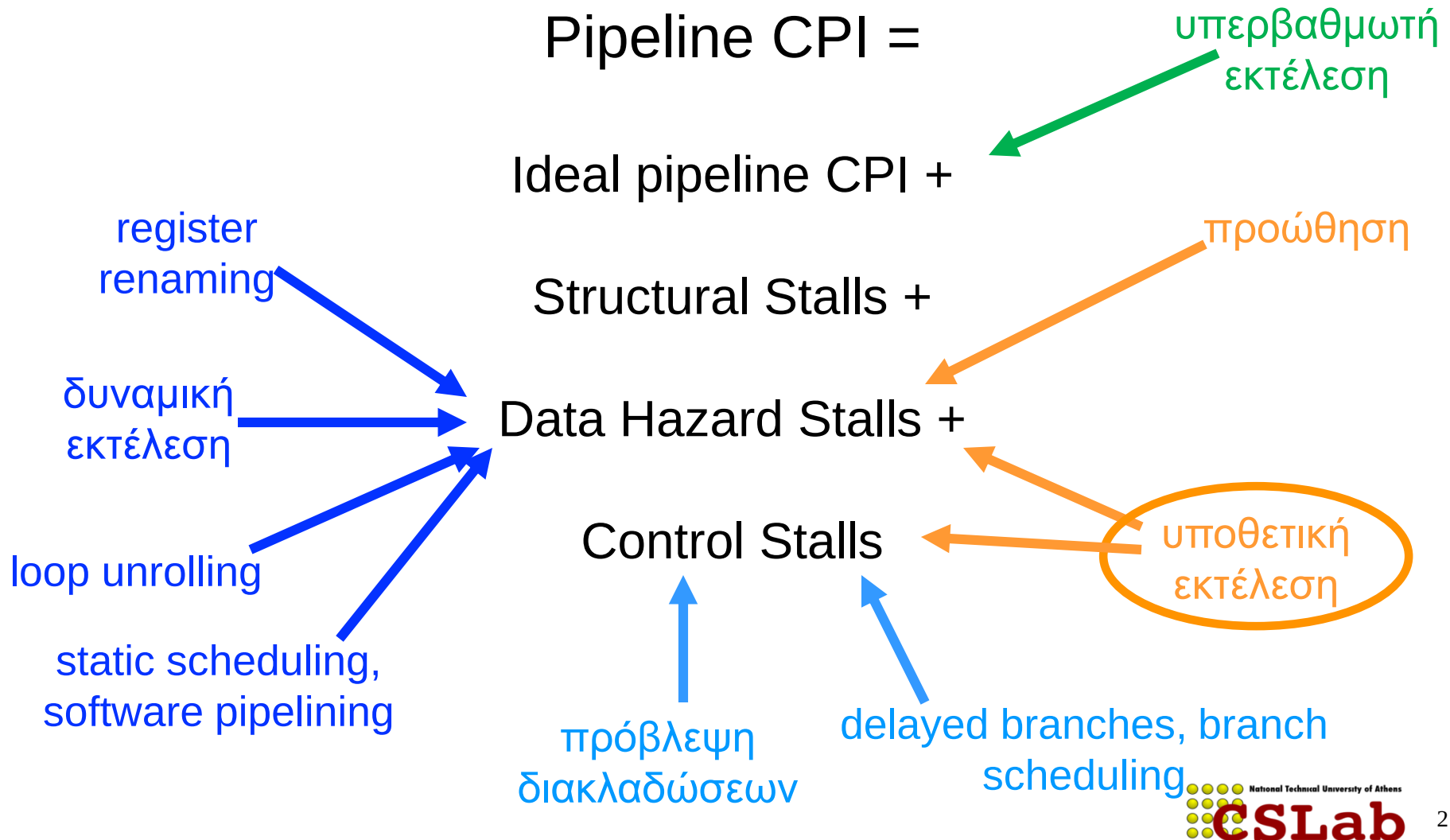
8ο εξάμηνο ΣΗΜΜΥ – Ακαδημαϊκό Έτος: 2025-26

<https://helios.ntua.gr/course/view.php?id=1039>

Figures are from:

- CA5e/Patterson-Hennessy (2012 Elsevier) and
- Modern Processor Design, Shen and Lipasti (2005 McGraw Hill, 2013 Waveland Press)
- Examples/Slides from cs252/152Berkeley Kubiatawicz, Patterson, Brodersen* / cs146Harvard D. Brooks

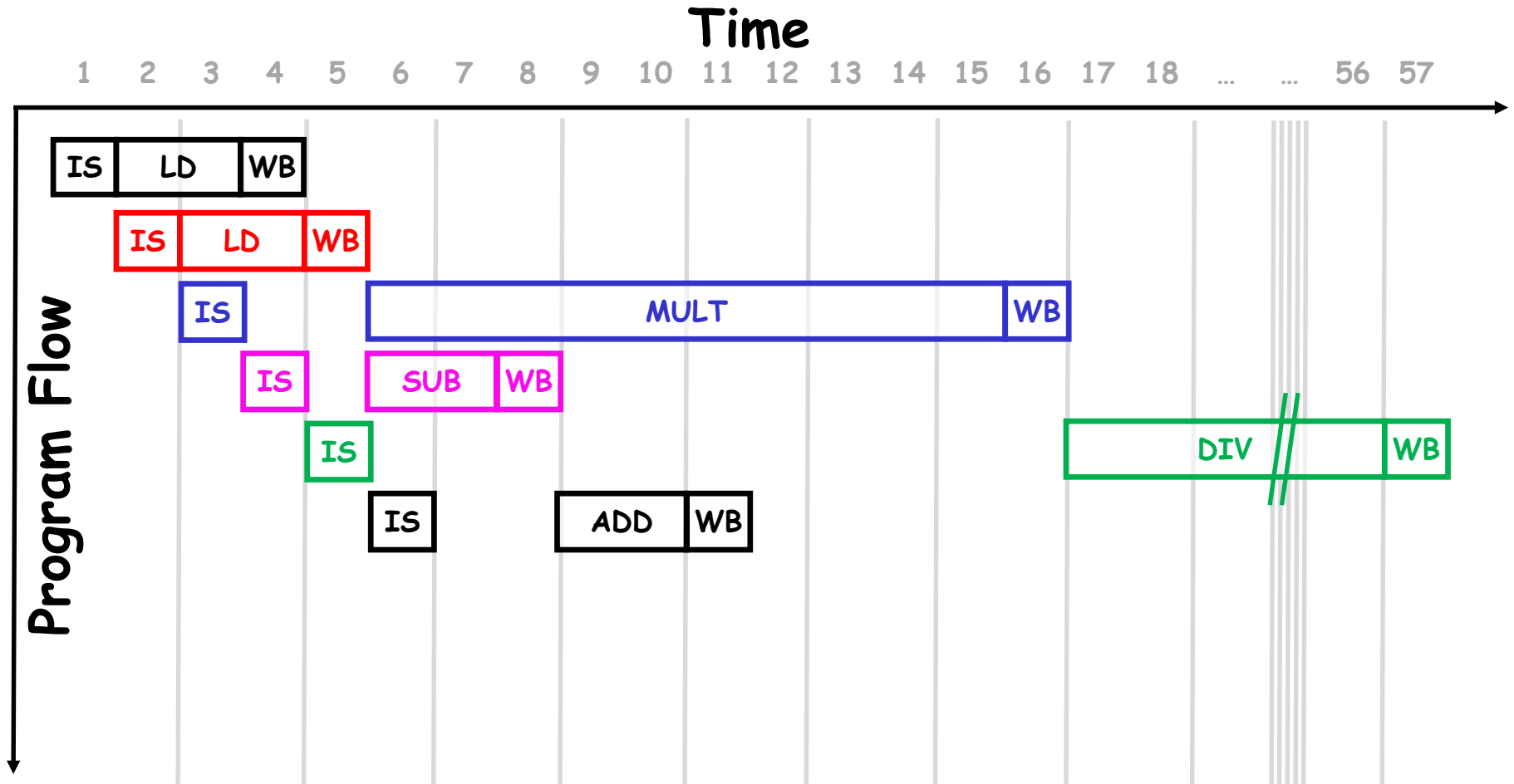
Τεχνικές βελτίωσης του CPI



Δυναμική Δρομολόγηση Εντολών

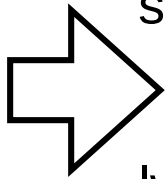
- Tomasulo, Explicit Register Renaming
 - in-order issue
 - out-of-order execution
 - out-of-order completion
- Περισσότερος παραλληλισμός (ILP)
- Βελτίωση της απόδοσης του συστήματος
- Προβλήματα
 - Interrupts/Exceptions
 - Εντολές διακλάδωσης (branches)

Tomasulo Out of Order Completion



Device Interrupt

External Interrupt



```
add    r1,r2,r3
subi   r4,r1,#4
slli   r4,r4,#2
```

(!)

```
lw     r2,0(r4)
lw     r3,4(r4)
add    r2,r2,r3
sw     8(r4),r2
```

save PC
disable Ints
Supervisor Mode

Interrupt Handler

increase priority

De-activate Interrupts

save registers

```
lw  r1,20(r0)
```

```
lw  r2,0(r1)
```

```
addi r3,r0,#5
```

```
sw  0(r1),r3
```

restore registers

clear Interrupt status on device

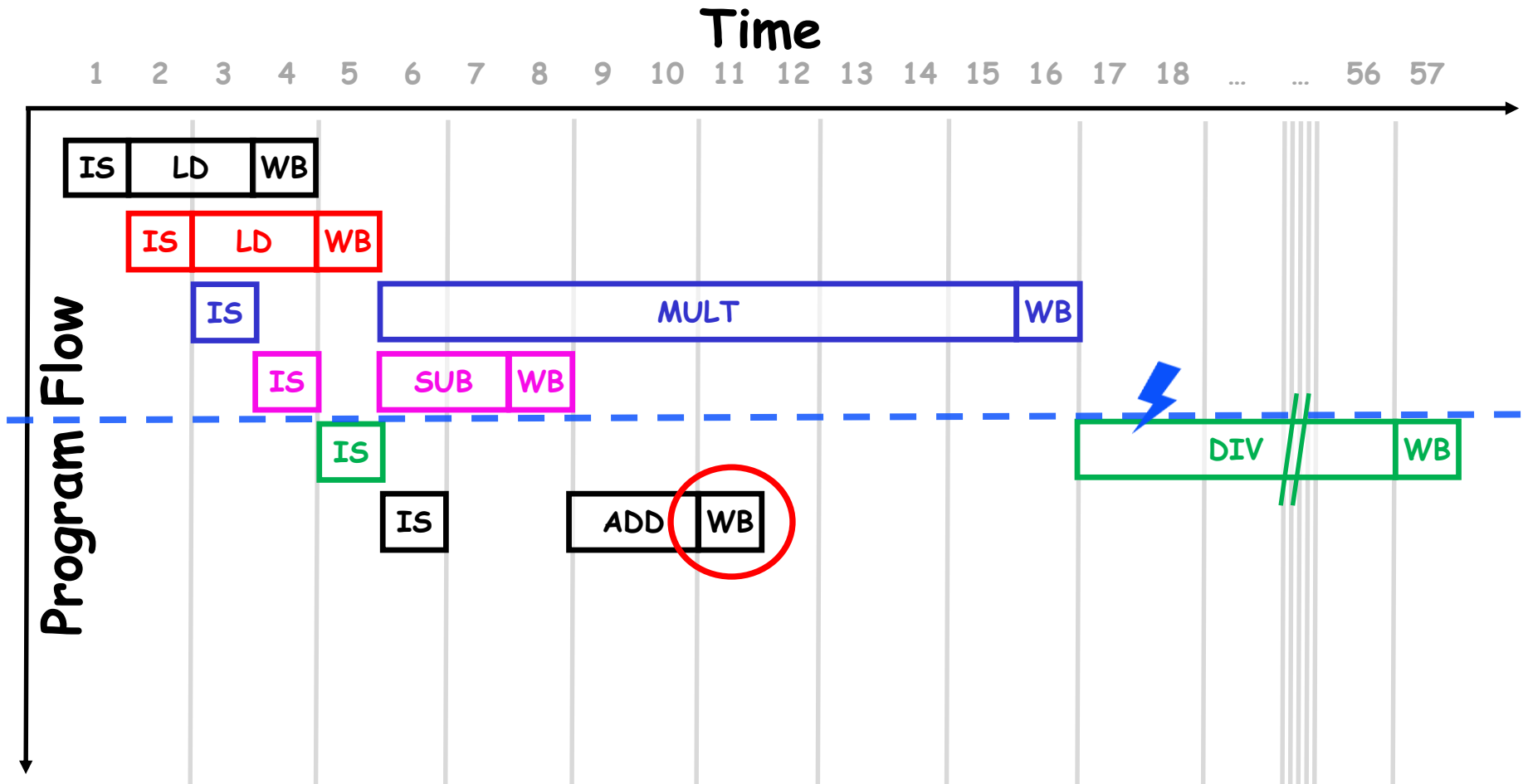
Re-activate Ints (if run w/
disabled)

restore priority

```
eret
```

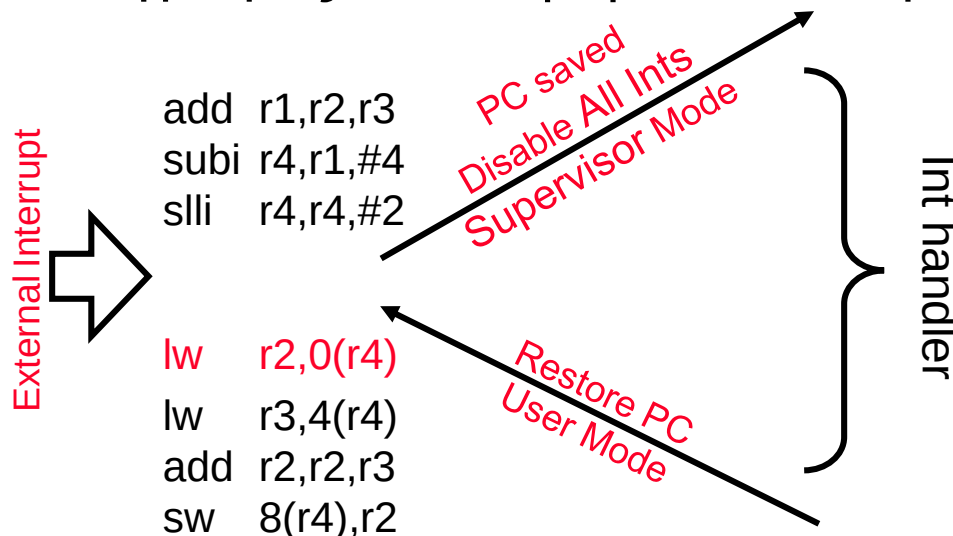
Restore PC
User Mode

Που είναι το πρόβλημα?



Precise Interrupts/Exceptions

- Ένα interrupt ή exception ονομάζεται **precise** εάν υπάρχει μία εντολή (ή interrupt point) για το οποίο:
 - Όλες οι προηγούμενες εντολές έχουν πλήρως εκτελεστεί
 - Καμία »επόμενη» εντολή (μαζί με την interrupting instruction) δεν έχει αλλάξει την κατάσταση της μηχανής
- Αυτό σημαίνει ότι μπορούμε να επανεκκινήσουμε την εκτέλεση από το interrupt point και “να πάρουμε τα σωστά αποτελέσματα”
 - Στο παράδειγμά μας: Interrupt point είναι η **lw** εντολή



Γιατί χρειαζόμαστε precise interrupts?

- Αρκετά interrupts/exceptions πρέπει να είναι restartable
 - π.χ. TLB faults: διόρθωση translation και επανάληψη του load/store
 - IEEE underflows, illegal operations
- Η επανεκκίνηση δεν απαιτεί «preciseness». Ωστόσο, με «preciseness» είναι **πολύ πιο εύκολη!**
- Απλοποιεί το λειτουργικό σύστημα
 - το process state που χρειάζεται να αποθηκευτεί είναι μικρότερο
 - η επανεκκίνηση γίνεται γρήγορα (καλό για interrupts μικρής διάρκειας/μεγάλης συχνότητας)

Πού είναι το πρόβλημα;

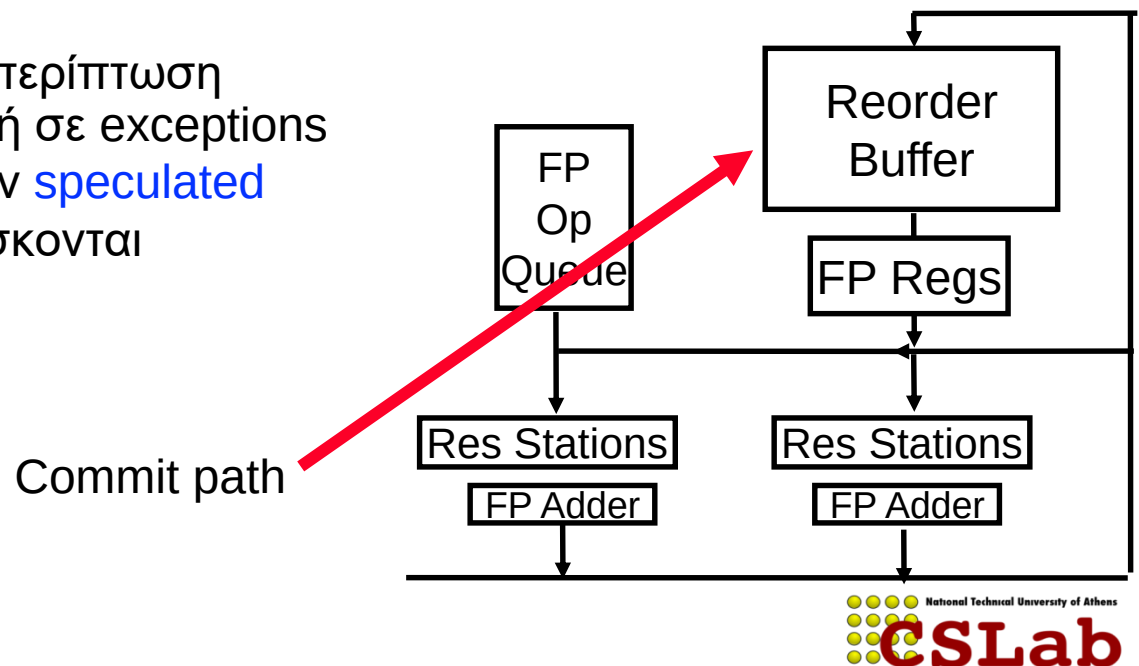
- Όλες οι τεχνικές που είδαμε μέχρι τώρα (Tomasulo, Explicit Register Renaming) υλοποιούν **out-of-order completion**
 - **Imprecise** και όχι **Precise** Interrupts/Exceptions
 - » Είναι πιθανό όταν εμφανισθεί ένα Interrupt/Exception η κατάσταση της μηχανής (processor state) να είναι διαφορετική από ότι αν οι εντολές είχαν εκτελεστεί σειριακά 
- Πρέπει να αλλάξουμε τη λογική του out-of-order completion, ώστε να μπορούμε να ορίζουμε precise interrupt points μέσα στο instruction stream
- Πρέπει να βρούμε τρόπο ώστε να *συγχρονίσουμε* το completion στάδιο των εντολών με την σειρά στο πρόγραμμα (issue-order)
 - Ο απλούστερος τρόπος: **in-order completion**

Εκτός από interrupts: Εντολές διακλάδωσης (branches)

- Για να βελτιώσουμε την απόδοση του συστήματος χρησιμοποιούμε **branch prediction**
- Επομένως κατά το οοο execution εκτελούμε εντολές οι οποίες εξαρτώνται από το αποτέλεσμα της εντολής διακλάδωσης
- Αν η πρόβλεψη δεν επαληθευτεί, θα πρέπει να κάνουμε rollback στο σημείο όπου κάναμε την πρόβλεψη, διότι οι εντολές στο λανθασμένο predicted path έχουν ήδη εκτελεστεί
 - Ιδιο το πρόβλημα με τα precise exceptions!
- Λύση: **in-order completion**

Hardware Support - ROB

- Ιδέα του **Reorder Buffer** (ROB):
 - Οι εντολές μπαίνουν σε μία FIFO, ακριβώς με την σειρά που γίνονται issue.
 - » Κάθε ROB entry περιέχει: instruction type (branch/store/register op), destination, value, ready field
 - Όταν η εντολή τελειώσει την εκτέλεση, τοποθέτησε τα αποτελέσματα στον ROB.
 - » Παρέχει operands σε άλλες εντολές στο διάστημα μεταξύ execution complete & commit
 - Η εντολή αλλάζει την κατάσταση της μηχανής στο **commit στάδιο** όχι στο WB → in-order commit → οι τιμές στην κεφαλή του ROB αποθηκεύονται στο register file
 - Εύκολη η αναίρεση σε περίπτωση mispredicted branches ή σε exceptions
 - » Απλά **διαγραφή** των **speculated instructions** που βρίσκονται στο ROB



Αλγόριθμος Tomasulo με ROB

Issue — Πάρε εντολή από FP Op Queue

Αν υπάρχει ελεύθερο reservation station & reorder buffer entry, issue instr & send operands & reorder buffer no. for destination

Execution — Εκτέλεσε εντολή στο Ex Unit (EX)

Όταν και οι τιμές και των 2 source regs είναι έτοιμες εκτέλεσε την εντολή; Αν όχι, παρακολούθησε το CDB για το αποτέλεσμα; Όταν και οι 2 τιμές βρίσκονται στο RS, εκτέλεσε την εντολή; (checks RAW)

Write result — Τέλος εκτέλεσης (WB)

Γράψε το αποτέλεσμα στο CDB προς όλες τις μονάδες που το περιμένουν & στον reorder buffer; Σημείωσε τον RS ως διαθέσιμο.

Commit — Ανανέωσε τον dest register με το αποτέλεσμα από τον reorder buffer

Όταν η εντολή φτάσει στην κεφαλή του reorder buffer & το result είναι διαθέσιμο, ανανέωσε τον dest register με αυτό (ή, αντίστοιχα, αποθήκευσε στη μνήμη) και βγάλε την εντολή από τον reorder buffer. Όταν η εντολή είναι mispredicted branch, καθάρισε (flush) τον reorder buffer και επανεκκίνησε την εκτέλεση στο σωστό path.

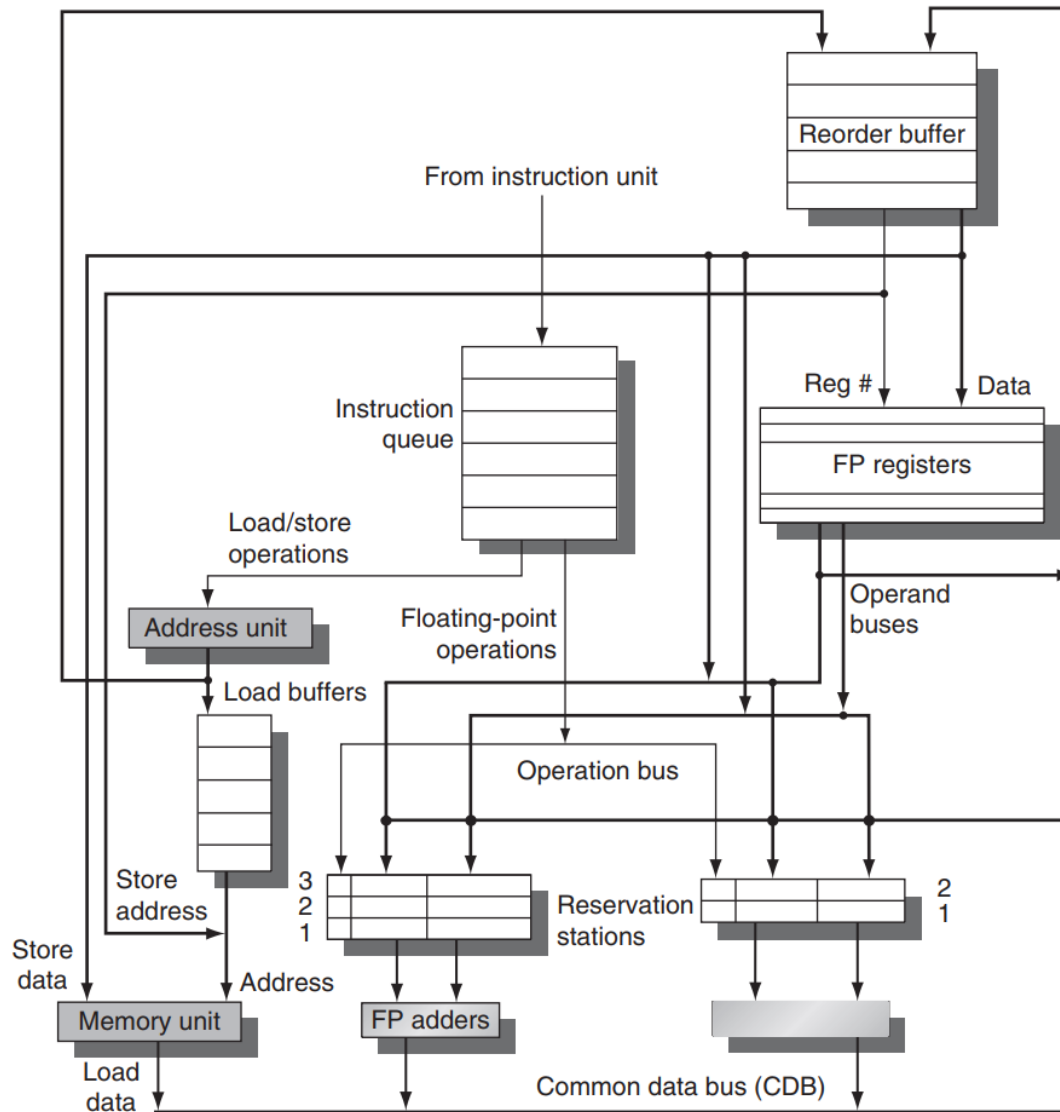
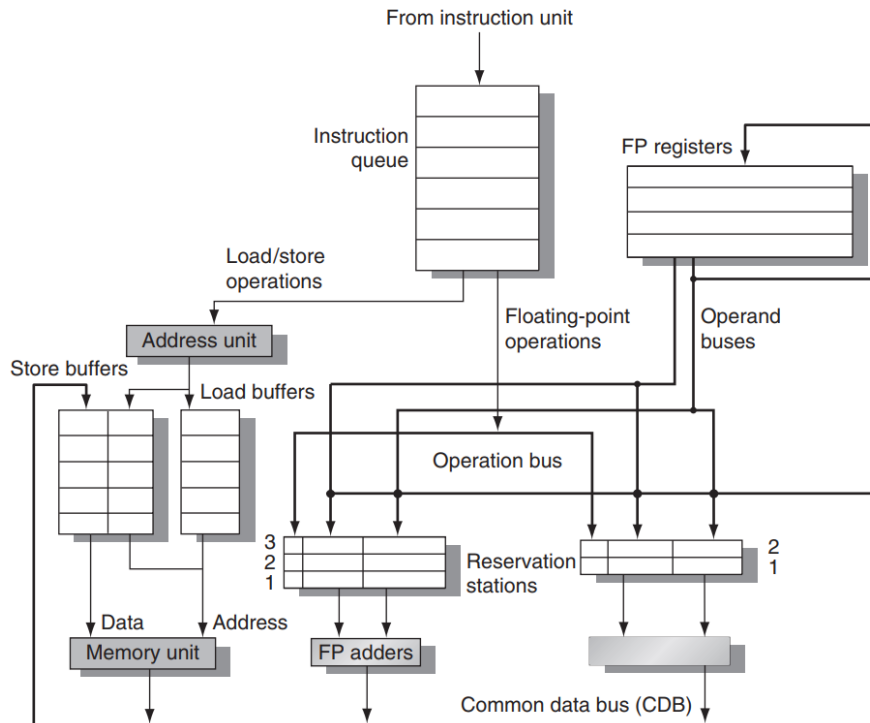
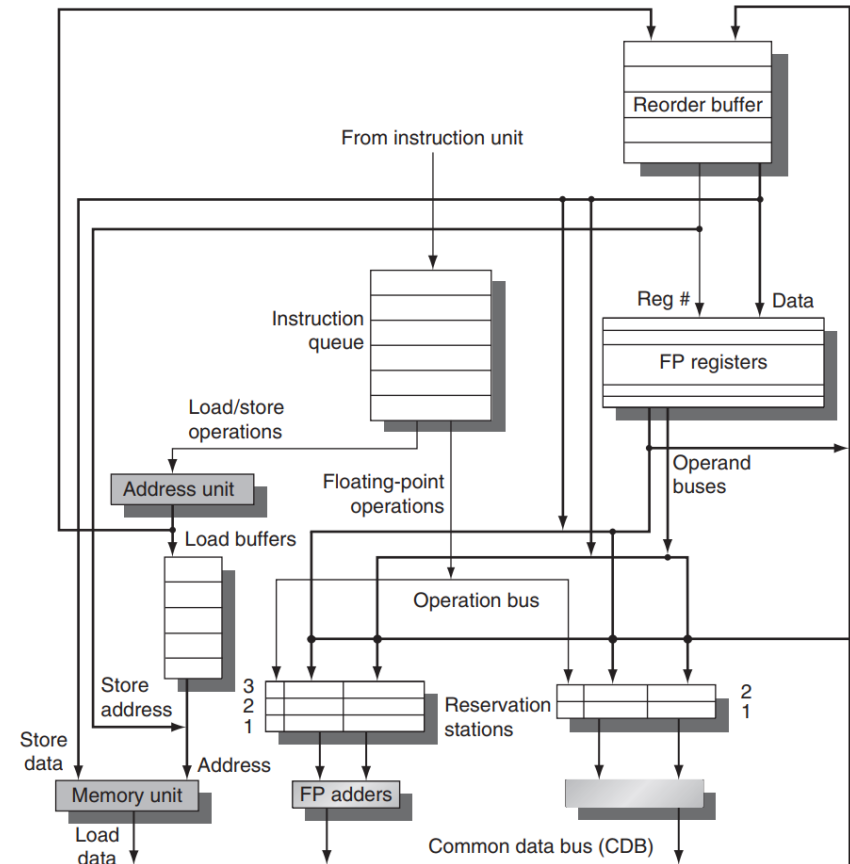


Figure 3.11 The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation. Comparing this to Figure 3.6 on page 173, which implemented Tomasulo's algorithm, the major change is the addition of the ROB and the elimination of the store buffer, whose function is integrated into the ROB. This mechanism can be extended to multiple issue by making the CDB wider to allow for multiple completions per clock.

Dynamic Scheduling with Tomasulo



Dynamic Scheduling with speculation (ROB)



Tomasulo With Reorder buffer - Issue

Status	Wait until	Action or bookkeeping
Issue all instructions	Reservation station (r) and ROB (b) both available	<pre> if (RegisterStat[rs].Busy) /*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready) /* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no; </pre>
FP operations and stores		<pre> if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready) /* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;}; </pre>
FP operations		<pre> RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd; </pre>
Loads		<pre> RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt; </pre>
Stores		<pre> RS[r].A ← imm; </pre>

Tomasulo With Reorder buffer - Execute

Status	Wait until	Action or bookkeeping
Execute FP op	$(RS[r].Qj == 0)$ and $(RS[r].Qk == 0)$	Compute results—operands are in Vj and Vk
Load step 1	$(RS[r].Qj == 0)$ and there are no stores earlier in the queue	$RS[r].A \leftarrow RS[r].Vj + RS[r].A;$
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from $Mem[RS[r].A]$
Store	$(RS[r].Qj == 0)$ and store at queue head	$ROB[h].Address \leftarrow RS[r].Vj + RS[r].A;$

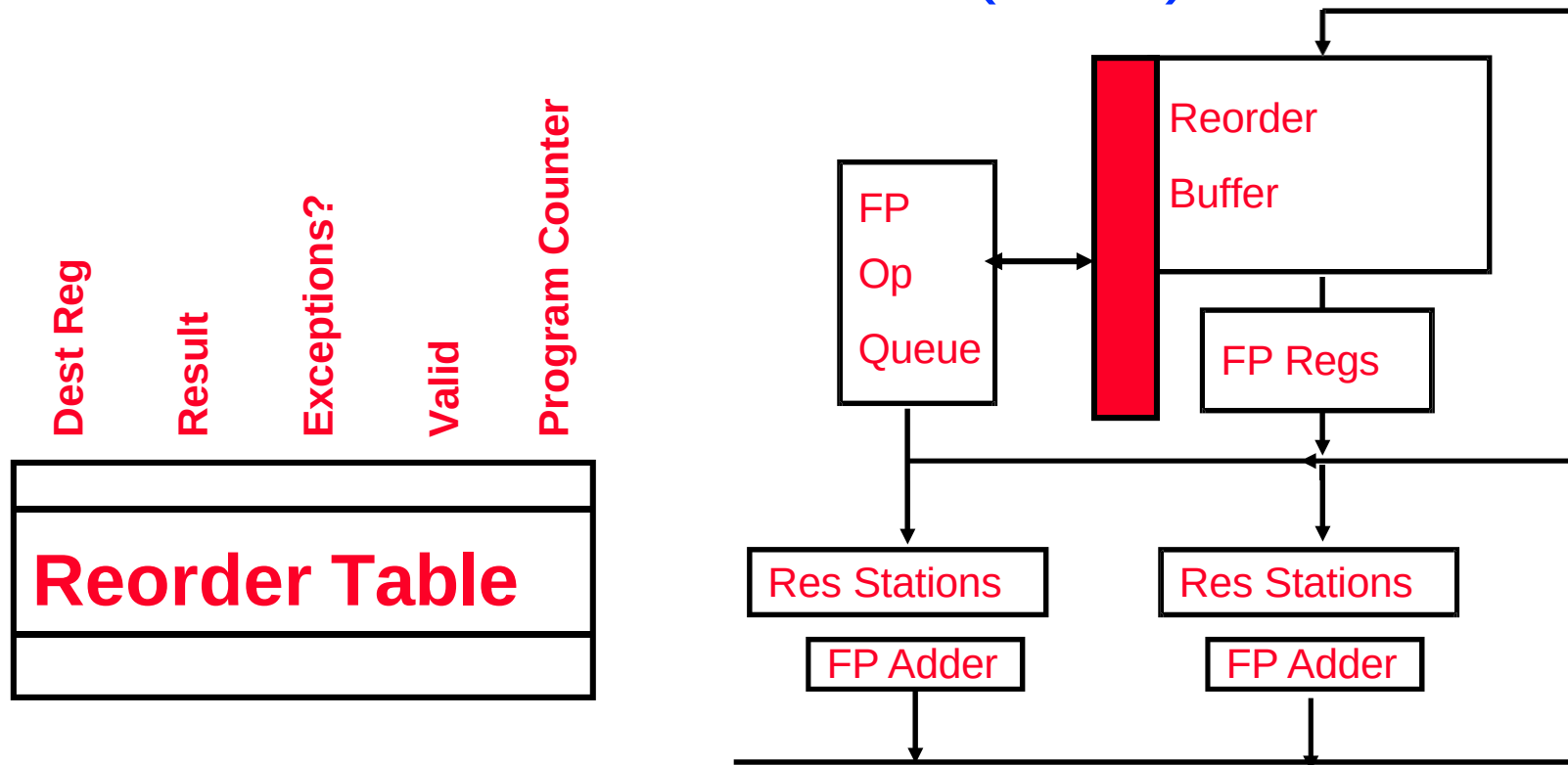
Tomasulo With Reorder buffer – WB/Commit

Status	Wait until	Action or bookkeeping
Write result all but store	Execution done at r and CDB available	$b \leftarrow RS[r].Dest; RS[r].Busy \leftarrow no;$ $\forall x (if (RS[x].Qj == b) \{RS[x].Vj \leftarrow result; RS[x].Qj \leftarrow 0\});$ $\forall x (if (RS[x].Qk == b) \{RS[x].Vk \leftarrow result; RS[x].Qk \leftarrow 0\});$ $ROB[b].Value \leftarrow result; ROB[b].Ready \leftarrow yes;$
Store	Execution done at r and $(RS[r].Qk == 0)$	$ROB[h].Value \leftarrow RS[r].Vk;$
Commit	Instruction is at the head of the ROB (entry h) and $ROB[h].ready == yes$	$d \leftarrow ROB[h].Dest; /* register dest, if exists */$ $if (ROB[h].Instruction == Branch)$ $\quad \{if (branch is mispredicted)$ $\quad \quad \{clear ROB[h], RegisterStat; fetch branch dest;\};\}$ $else if (ROB[h].Instruction == Store)$ $\quad \{Mem[ROB[h].Destination] \leftarrow ROB[h].Value;\}$ $else /* put the result in the register destination */$ $\quad \{Regs[d] \leftarrow ROB[h].Value;\};$ $ROB[h].Busy \leftarrow no; /* free up ROB entry */$ $/* free up dest register if no one else writing it */$ $if (RegisterStat[d].Reorder == h) \{RegisterStat[d].Busy \leftarrow no;\};$

Tomasulo with Speculation: 4-step execution

- 1) **Issue**—get instruction from instruction queue—among the instructions, there will be branch instructions and **target instructions** (of course with predicted branch)—These target instructions(along with Store) will go into ROB
If a *reservation station* and *reorder buffer slot* free, issue instr & send(to RS) operands & reorder buffer # (for RS to get the operand when it(#) commits)
- 2) **Execute**
When both operands ready then execute; if not ready, watch CDB for result;
when both operands are in reservation station, execute; checks for RAW
Cycles taken are dependent on the operation—Load=2 cycles, Store=1 cycle,
- 3) **Write result**—finish execution (WB)
Write on Common Data Bus to all awaiting RSs & reorder buffer; mark reservation station as available(free). (tags are now ROB #s not RS #s)
- 4) **Commit**—update register with reorder result
Three sequences of actions when an instruction reaches the head of ROB:
 - Normal Commit: write registers, in-order commit
 - Store: update memory
 - Branch with incorrect prediction: flush ROB and flush reservation stations and restart execution at correct PC`

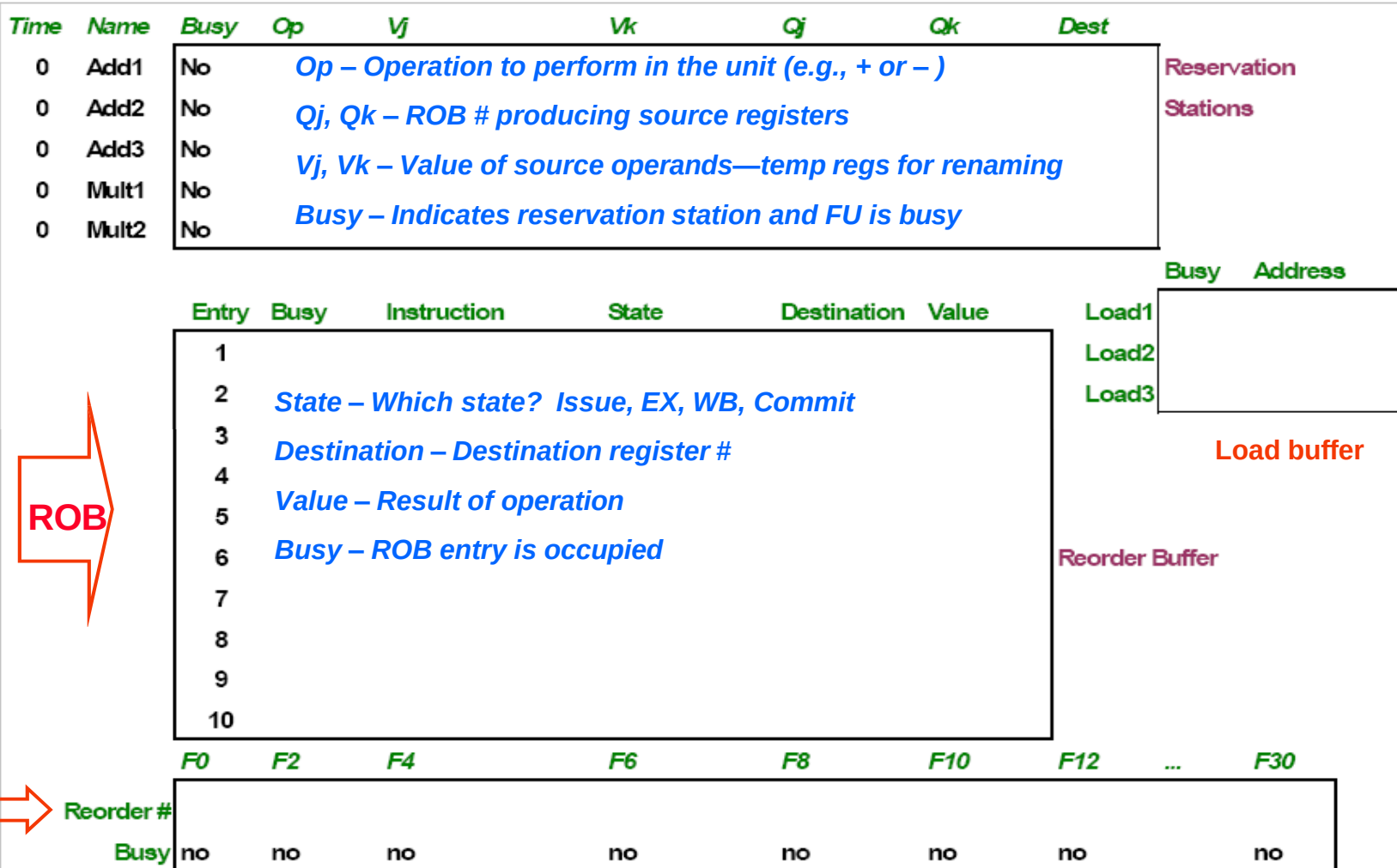
What are the hardware complexities with reorder buffer (ROB)?



- How do you find the latest version of a register?
 - need associative comparison network
 - Could use future file or just use the register result status buffer to track which specific reorder buffer has received the value
- Need as many ports on ROB as register file

- If a branch is **mispredicted**, recovery is done by flushing the ROB of all entries that appear after the mispredicted branch
 - entries before the branch are allowed to continue
 - restart the fetch at the correct branch successor
- When an instruction commits or is flushed from the ROB then the corresponding slots become available for subsequent instructions

Tomasulo With Reorder Buffer - Cycle 0



ROB

Load buffer

Reorder Buffer

Example

LD F6, 34(R2)

LD F2, 45(R3)

MULT F0, F2, F4

SUBD F8, F6, F2

DIVD F10, F0, F6

ADDD F6, F8, F2

Tomasulo With Reorder Buffer - Cycle 1

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	No						
0	Mult2	No						

Reservation Stations

Entry	Busy	Instruction	State	Destination	Value
1	Yes	LD F6, 34(R2)	Issue	F6	
2					
3					
4					
5					
6					
7					
8					
9					
10					

Reorder Buffer

Reorder #	Busy	F0	F2	F4	F6	F8	F10	F12	...	F30
	no	no	no		#1					
	no	no	no		Yes	no	no	no		no

Register Renaming!

Load1	Busy	Address
Load1	Yes	34+Regs[R2]
Load2		
Load3		

Tomasulo With Reorder Buffer - Cycle 2

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	No						
0	Mult2	No						

Reservation Stations

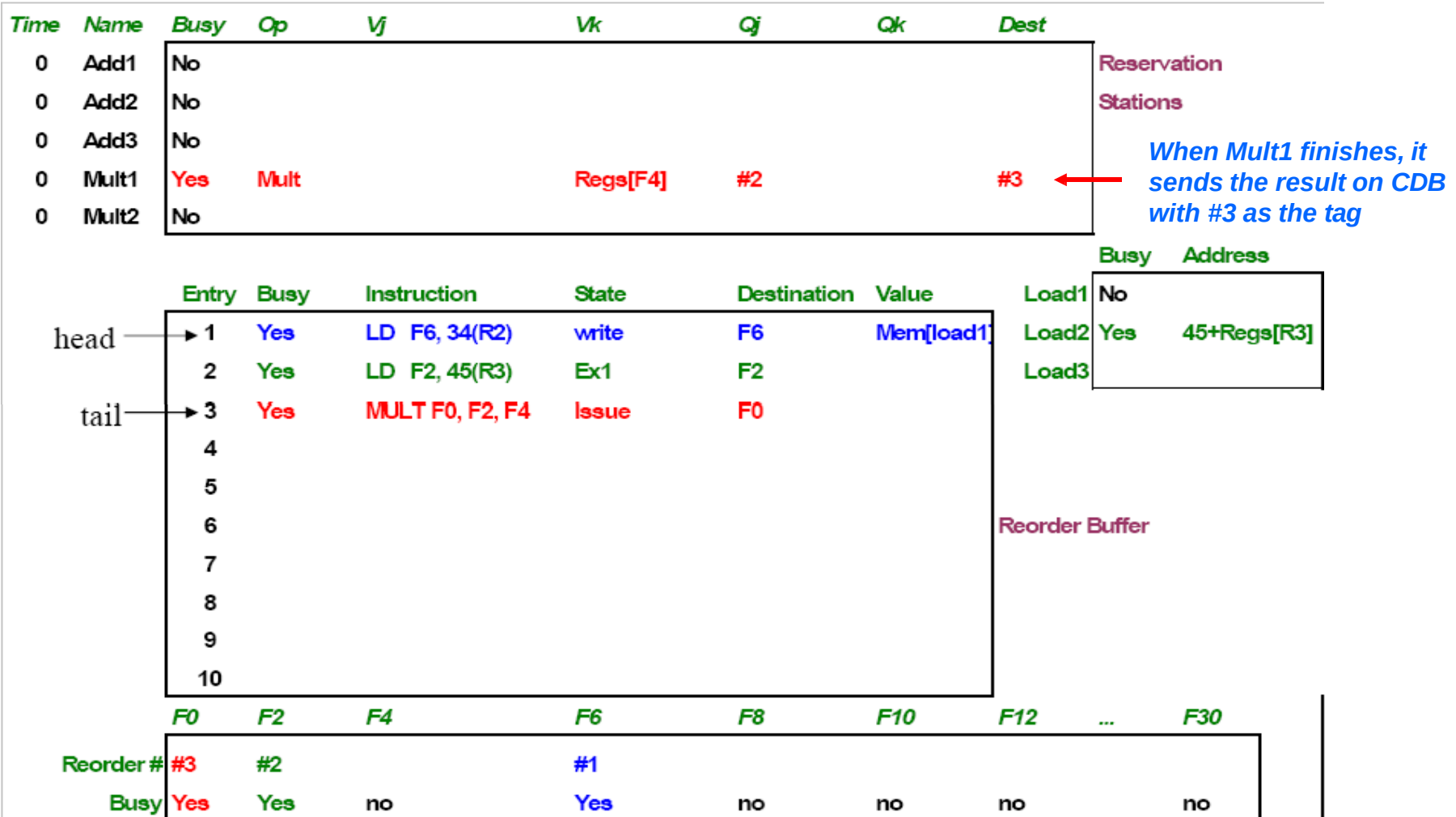
Entry	Busy	Instruction	State	Destination	Value
head → 1	Yes	LD F6, 34(R2)	Ex1	F6	
tail → 2	Yes	LD F2, 45(R3)	Issue	F2	
3					
4					
5					
6					
7					
8					
9					
10					

Reorder Buffer

Reorder #	F0	F2	F4	F6	F8	F10	F12	...	F30
#		#2		#1					
Busy	no	Yes	no	Yes	no	no	no		no

Load1	Busy	Address
Load1	Yes	34+Regs[R2]
Load2	Yes	45+Regs[R3]
Load3		

Tomasulo With Reorder Buffer - Cycle 3



Tomasulo With Reorder Buffer - Cycle 4

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	Yes	SUB	Regs[F6]	Mem[45+Regs[R3]]			#4	Reservation Stations
0	Add2	No							
0	Add3	No							
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3	
0	Mult2	No							

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No
2	Yes	LD F2, 45(R3)	write	F2	Mem[load2]	No	No	No
3	Yes	MULT F0, F2, F4	EX1	F0				
4	Yes	SUBD F8, F6, F2	Issue	F8				
5								
6								
7								
8								
9								
10								

head → 2
tail → 4

For SUBD, the Control logic checks F2 and finds that it was renamed to #2, so look at the CDB for #2(which is broadcasted at this moment) and gets the data—Mem[45+Regs[R3]]

Reorder Buffer

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3	#2			#4				
Busy	Yes	Yes	no	no	Yes	no	no		no

Tomasulo With Reorder Buffer - Cycle 5

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	Reservation Stations	
0	Add1	Yes	SUB	Regs[F6]	Mem[45+Regs[R3]]			#4		
0	Add2	No								
0	Add3	No								
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3		
0	Mult2	Yes	DIV		Regs[F6]	#3		#5		

		Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
		1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
		2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
head	→	3	Yes	MULT F0, F2, F4	Ex2	F0						
		4	Yes	SUBD F8, F6, F2	Ex1	F8						
tail	→	5	Yes	DIVD F10, F0, F6	Issue	F10						
		6										
		7										
		8										
		9										
		10										

		F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3					#4	#5			
Busy	Yes	no	no	no	no	Yes	Yes	no		no

Reorder Buffer

Tomasulo With Reorder Buffer - Cycle 6

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	Yes	SUB	Regs[F6]	Mem[45+Regs[R3]]			#4	Reservation
0	Add2	Yes	Add		Regs[F2]	#4		#6	Stations
0	Add3	No							
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3	
0	Mult2	Yes	DIV		Regs[F6]	#3		#5	

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
3	Yes	MULT F0, F2, F4	Ex3	F0						
4	Yes	SUBD F8, F6, F2	Ex2	F8						
5	Yes	DIVD F10, F0, F6	Issue	F10						
6	Yes	ADDD F6, F8, F2	Issue	F6						
7										
8										
9										
10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 7

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	No							Reservation Stations
0	Add2	Yes	Add	#4	Regs[F2]			#6	
0	Add3	No							
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3	
0	Mult2	Yes	DIV		Regs[F6]	#3		#5	

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
3	Yes	MULT F0, F2, F4	Ex4	F0						
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2					
5	Yes	DIVD F10, F0, F6	Issue	F10						
6	Yes	ADDD F6, F8, F2	EX1	F6						
7										
8										
9										
10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 8

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	No							Reservation Stations
0	Add2	Yes	Add	#4	Regs[F2]			#6	
0	Add3	No							
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3	
0	Mult2	Yes	DIV		Regs[F6]	#3		#5	

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
3	Yes	MULT F0, F2, F4	Ex5	F0						
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2					
5	Yes	DIVD F10, F0, F6	Issue	F10						
6	Yes	ADDD F6, F8, F2	Ex2	F6						
7										
8										
9										
10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 9

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	No							Reservation Stations
0	Add2	Yes	Add	#4	Regs[F2]			#6	
0	Add3	No							
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3	
0	Mult2	Yes	DIV		Regs[F6]	#3		#5	

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
3	Yes	MULT F0, F2, F4	Ex6	F0						
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2					
5	Yes	DIVD F10, F0, F6	Issue	F10						
6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2					
7										
8										
9										
10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 10

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3
0	Mult2	Yes	DIV		Regs[F6]	#3		#5

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No
3	Yes	MULT F0, F2, F4	Ex7	F0				
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2			
5	Yes	DIVD F10, F0, F6	Issue	F10				
6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2			
7								
8								
9								
10								

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 11

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3
0	Mult2	Yes	DIV		Regs[F6]	#3		#5

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No
3	Yes	MULT F0, F2, F4	Ex8	F0				
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2			
5	Yes	DIVD F10, F0, F6	Issue	F10				
6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2			
7								
8								
9								
10								

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 12

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	Yes	Mult	Mem[45+Regs[R3]]	Regs[F4]			#3
0	Mult2	Yes	DIV		Regs[F6]	#3		#5

Reservation Stations

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No
3	Yes	MULT F0, F2, F4	Ex9	F0				
4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2			
5	Yes	DIVD F10, F0, F6	Issue	F10				
6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2			
7								
8								
9								
10								

Reorder Buffer

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 13

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	Reservation Stations	
0	Add1	No								
0	Add2	No								
0	Add3	No								
0	Mult1	No								
0	Mult2	Yes	DIV	#2xRegs[F4]	Regs[F6]			#5		

		Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
		1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No	No	
		2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No	No	
head	→	3	Yes	MULT F0, F2, F4	write	F0	#2 x Regs[F4]					
		4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2					
		5	Yes	DIVD F10, F0, F6	Ex1	F10						
tail	→	6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2					Reorder Buffer
		7										
		8										
		9										
		10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #	#3			#6	#4	#5			
Busy	Yes	no	no	Yes	Yes	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 14

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	Reservation Stations	
0	Add1	No								
0	Add2	No								
0	Add3	No								
0	Mult1	No								
0	Mult2	Yes	DIV	#2xRegs[F4]	Regs[F6]			#5		

		Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
		1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No	No	
		2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No	No	
		3	No	MULT F0, F2, F4	commit	F0	#2 x Regs[F4]					
head	→	4	Yes	SUBD F8, F6, F2	write	F8	F6 - #2					
		5	Yes	DIVD F10, F0, F6	Ex2	F10						
tail	→	6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2					
		7										
		8										
		9										
		10										

		F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #					#6	#4	#5			
Busy		No	no	no	Yes	Yes	Yes	no		no

Reorder Buffer

Tomasulo With Reorder Buffer - Cycle 15

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	Reservation Stations	
0	Add1	No								
0	Add2	No								
0	Add3	No								
0	Mult1	No								
0	Mult2	Yes	DIV	#2xRegs[F4]	Regs[F6]			#5		

		Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
		1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No	No	
		2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No	No	
		3	No	MULT F0, F2, F4	commit	F0	#2 x Regs[F4]					
		4	No	SUBD F8, F6, F2	commit	F8	F6 - #2					
head →		5	Yes	DIVD F10, F0, F6	Ex3	F10						
tail →		6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2					
		7										
		8										
		9										
		10										

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #				#6		#5			
Busy	no	no	no	Yes	no	Yes	no		no

Tomasulo With Reorder Buffer - Cycle 16

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	
0	Add1	No							Reservation Stations
0	Add2	No							
0	Add3	No							
0	Mult1	No							
0	Mult2	Yes	DIV	#2xRegs[F4]	Regs[F6]			#5	

Entry	Busy	Instruction	State	Destination	Value	Load1	Load2	Load3	Busy	Address
1	No	LD F6, 34(R2)	commit	F6	Mem[load1]	No	No	No		
2	No	LD F2, 45(R3)	commit	F2	Mem[load2]	No	No	No		
3	No	MULT F0, F2, F4	commit	F0	#2 x Regs[F4]					
4	No	SUBD F8, F6, F2	commit	F8	F6 - #2					
5	Yes	DIVD F10, F0, F6	Ex4	F10						
6	Yes	ADDD F6, F8, F2	write	F6	#4 + F2					
7										
8										
9										
10										

head → 5
tail → 6

Reorder Buffer

Need 36 more EX cycles for DIV to finish...

	F0	F2	F4	F6	F8	F10	F12	...	F30
Reorder #				#6		#5			
Busy	no	no	no	Yes	no	Yes	no		no

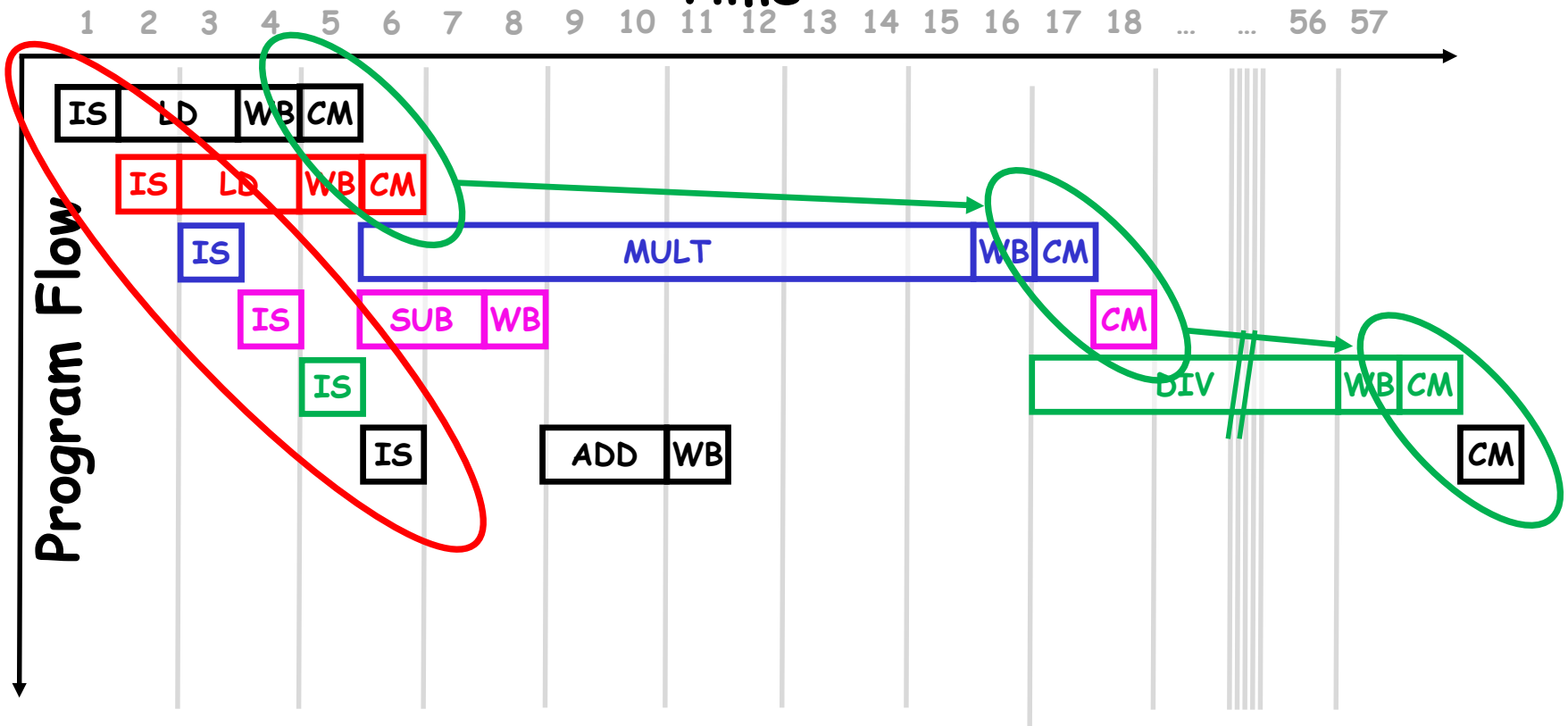
Summary: Tomasulo with ROB

Instruction	Issue	Exec Comp	Writeback	Commit
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	2	3	4	5
MULT F0, F2, F4	3	12	13	14
SUBD F8, F6, F2	4	6	7	15
DIVD F10, F0, F6	5	52	53	54
ADDD F6, F8, F2	6	8	9	55

In-order Issue/Commit, Out-of-Order Execution/Writeback

Tomasulo Dynamic Execution with ROB

Time

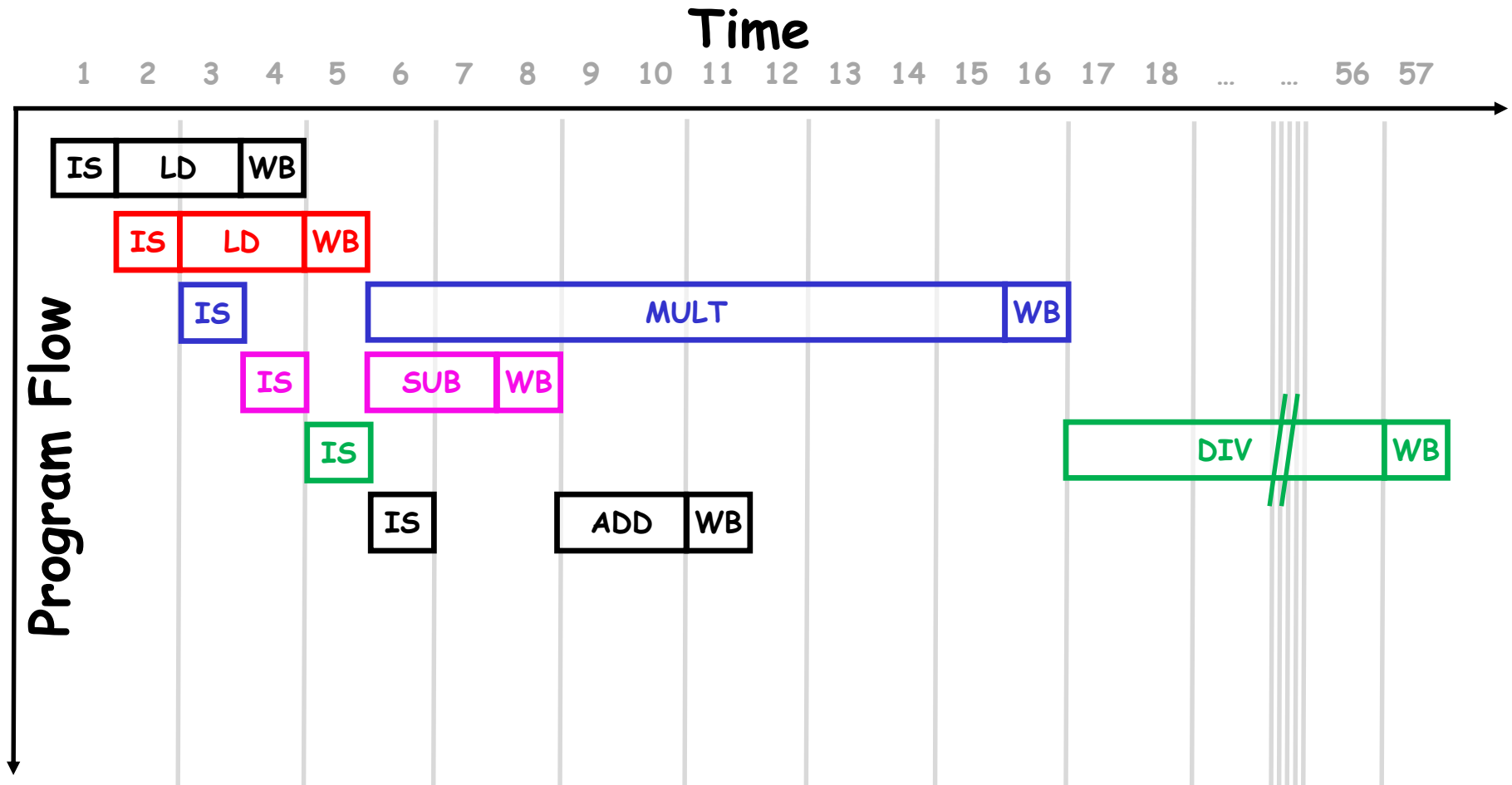


In-order issue

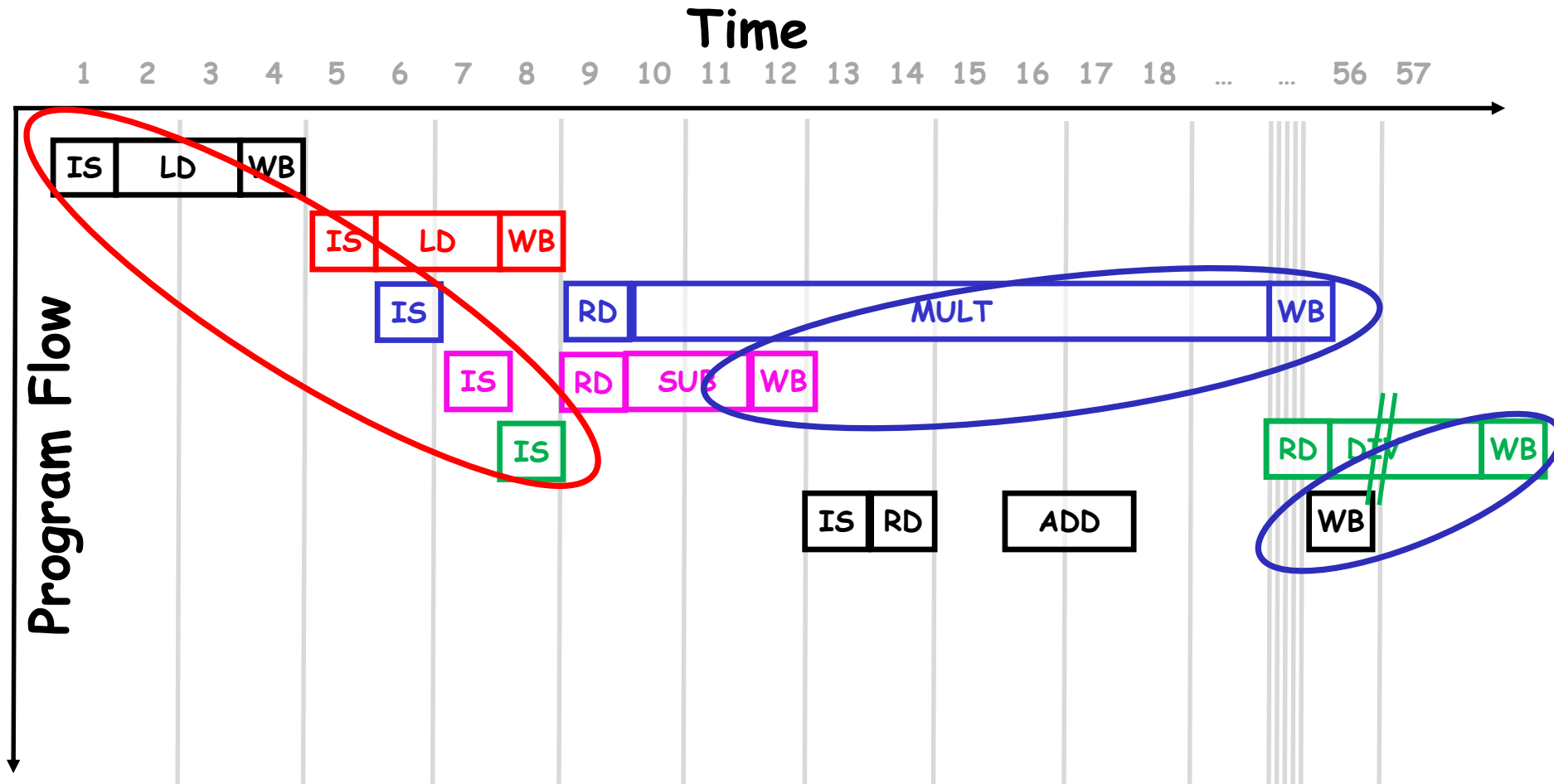
Out-of-order execution & result

In-order commit

Tomasulo Dynamic Execution (No ROB)



Scoreboard Dynamic Execution



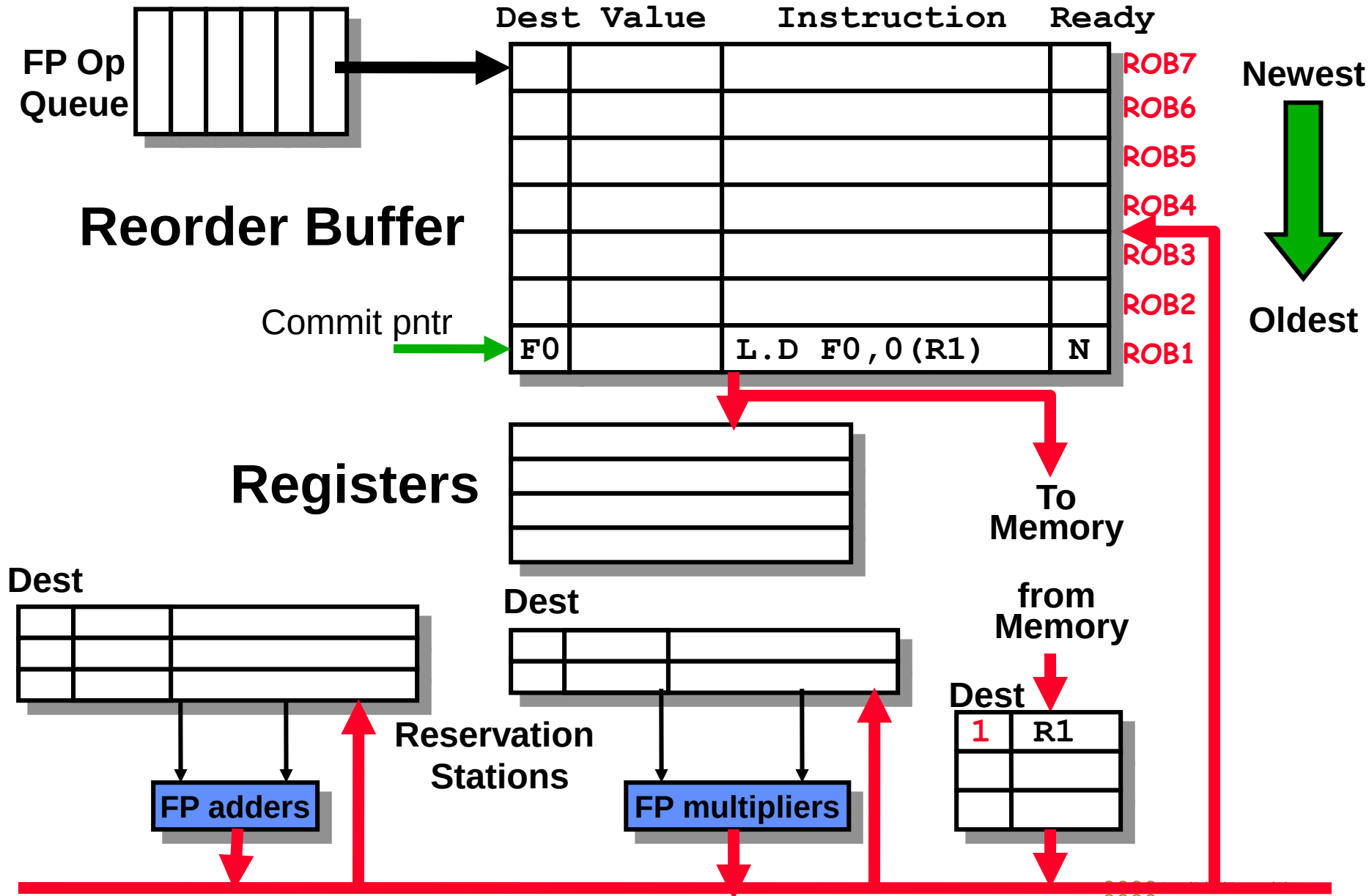
Tomasulo With Reorder buffer:

Loop example

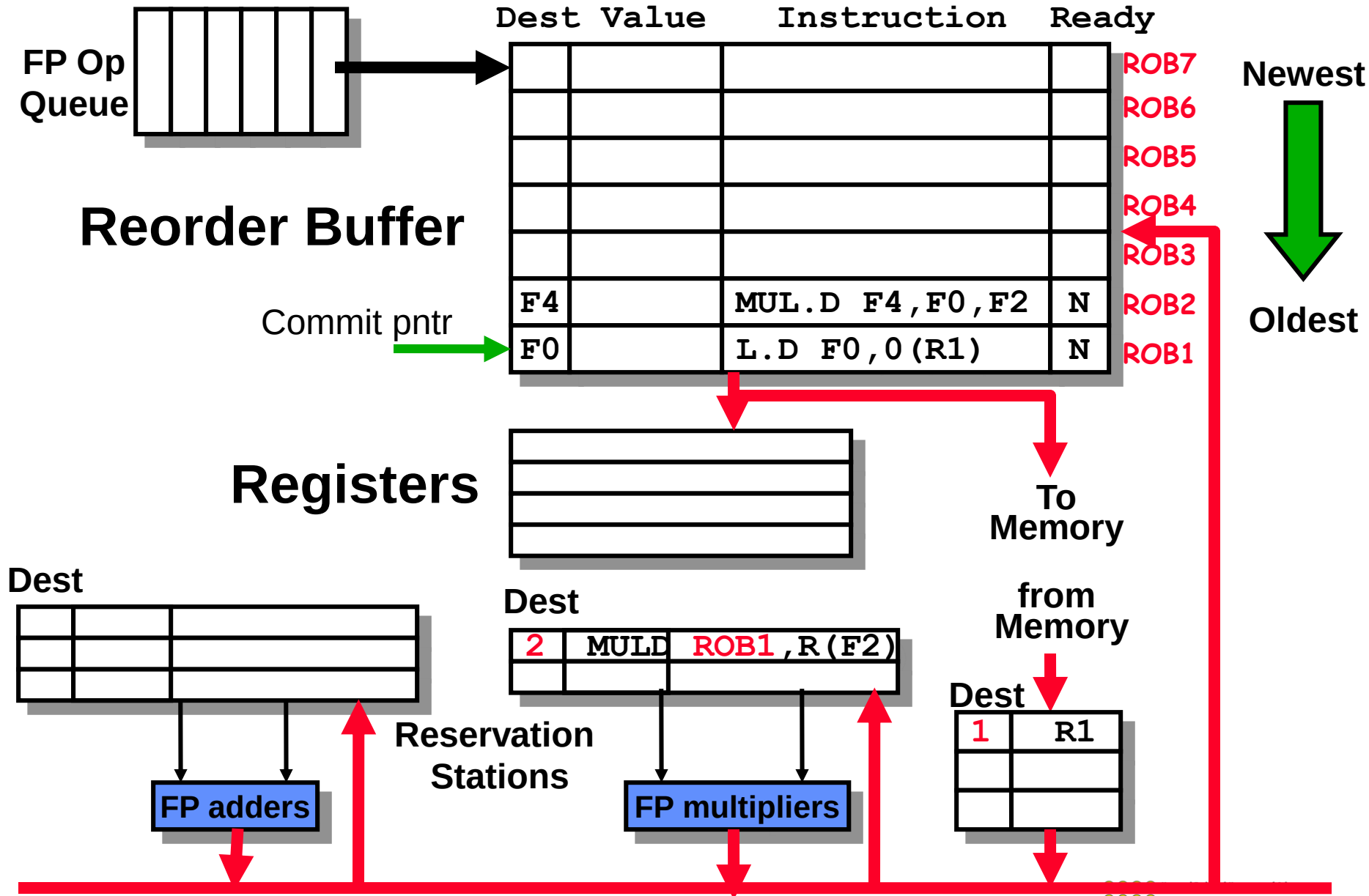
LOOP:

```
LD      F0,0(R1)
MULTD   F4,F0,F2
SD       F4,0(R1)
DADIU   R1, R1, -8
BNE     R1, R2, LOOP
```

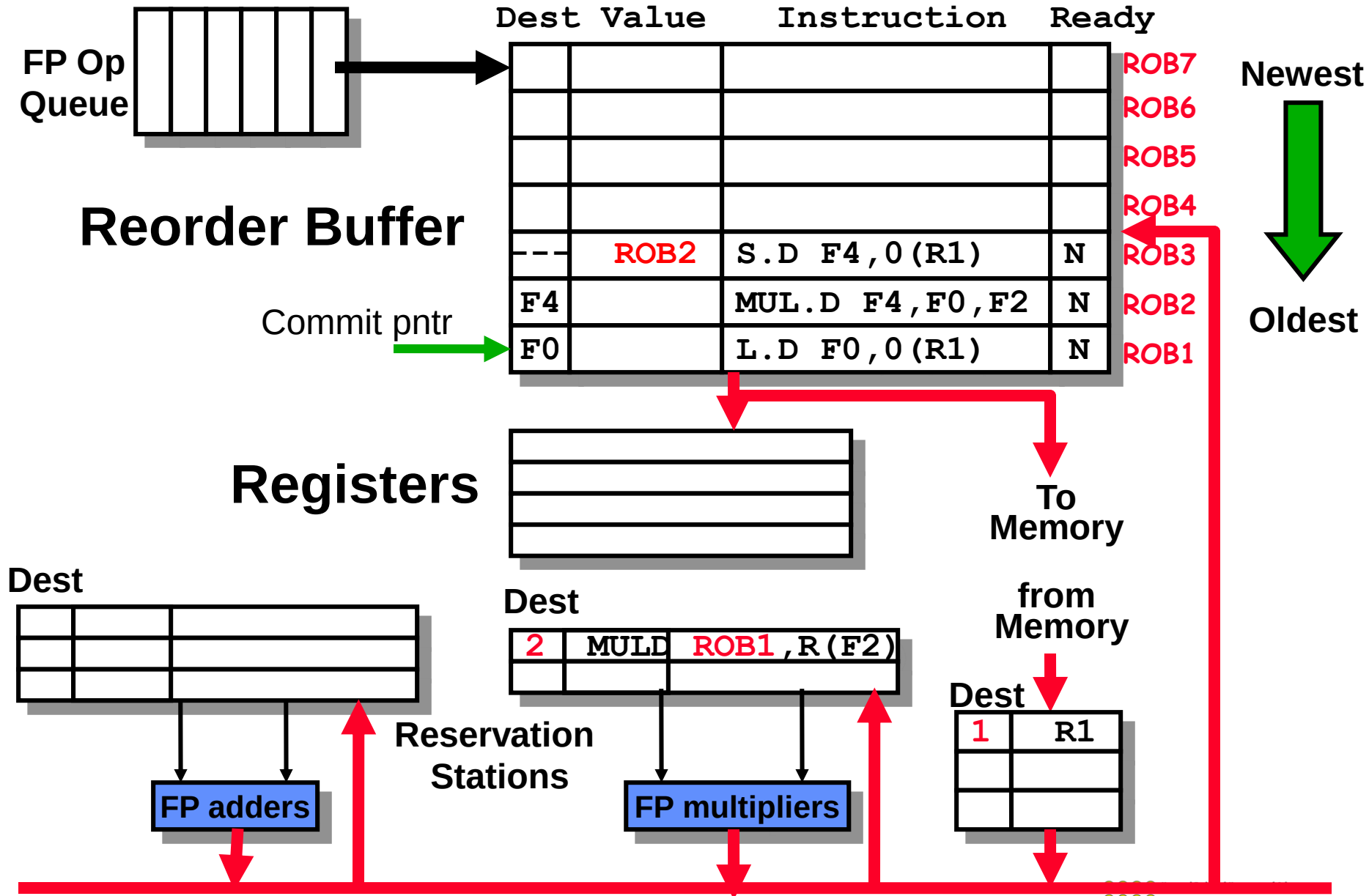
Tomasulo With Reorder buffer(1)



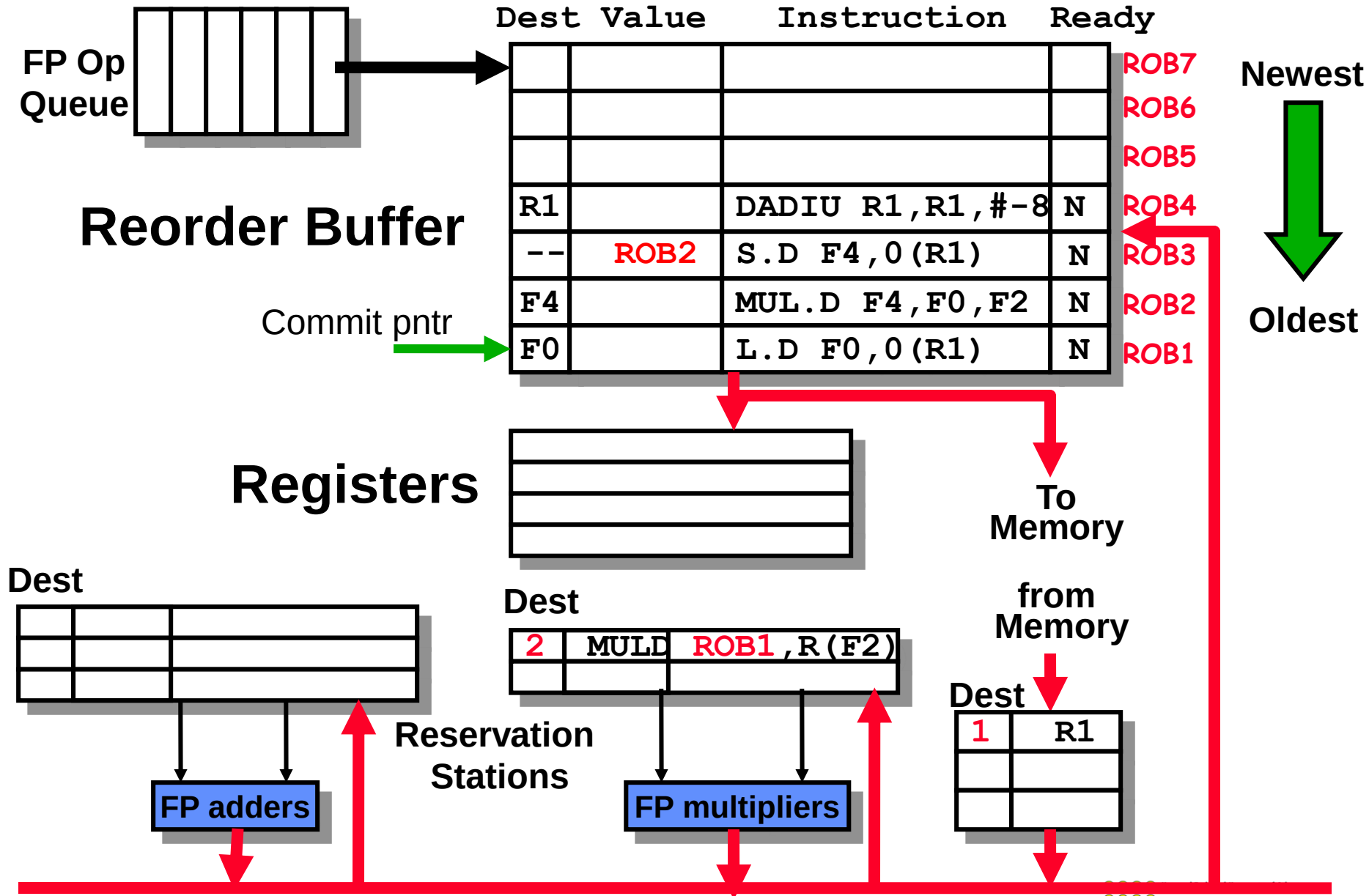
Tomasulo With Reorder buffer(2)



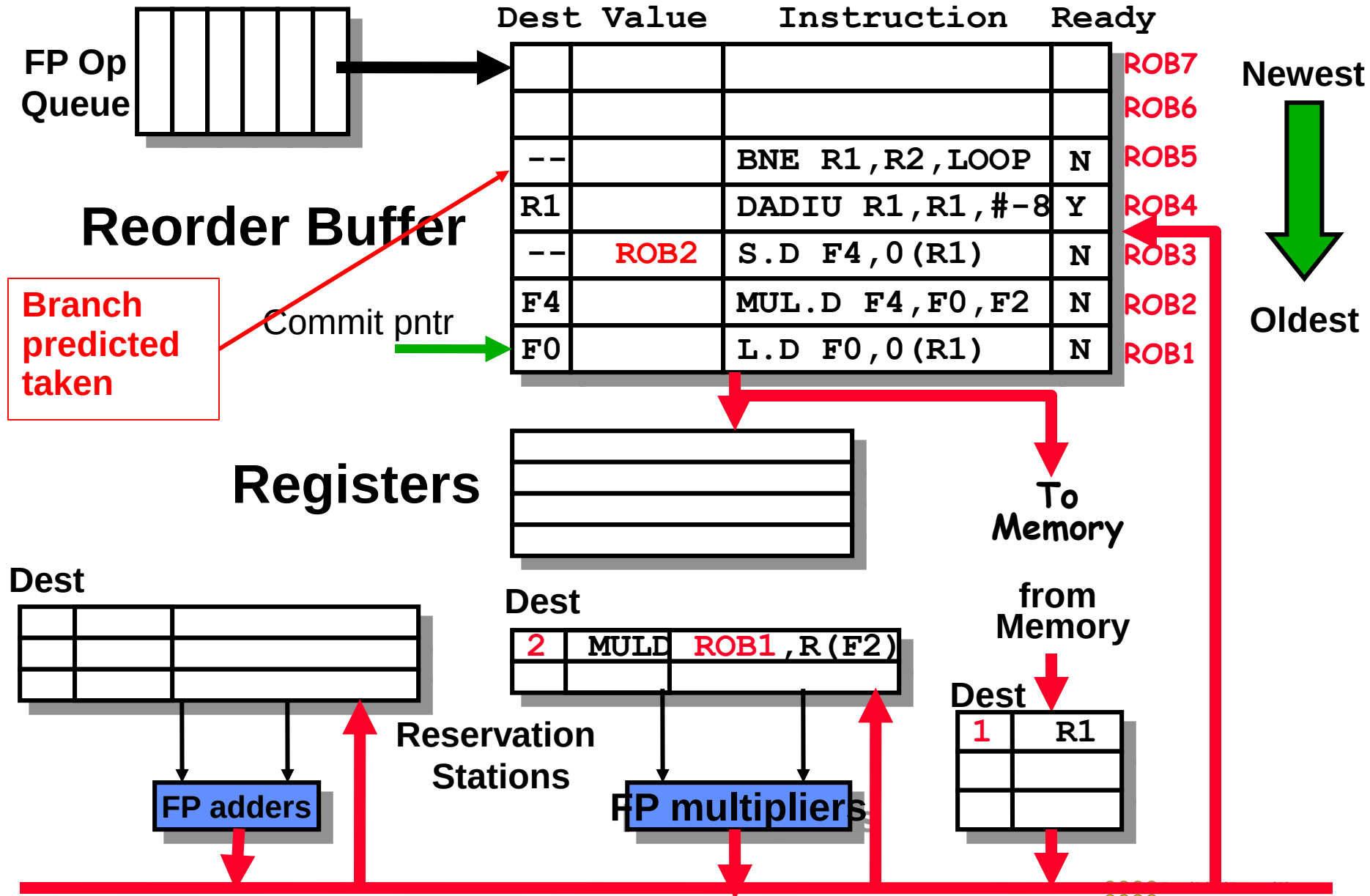
Tomasulo With Reorder buffer(3)



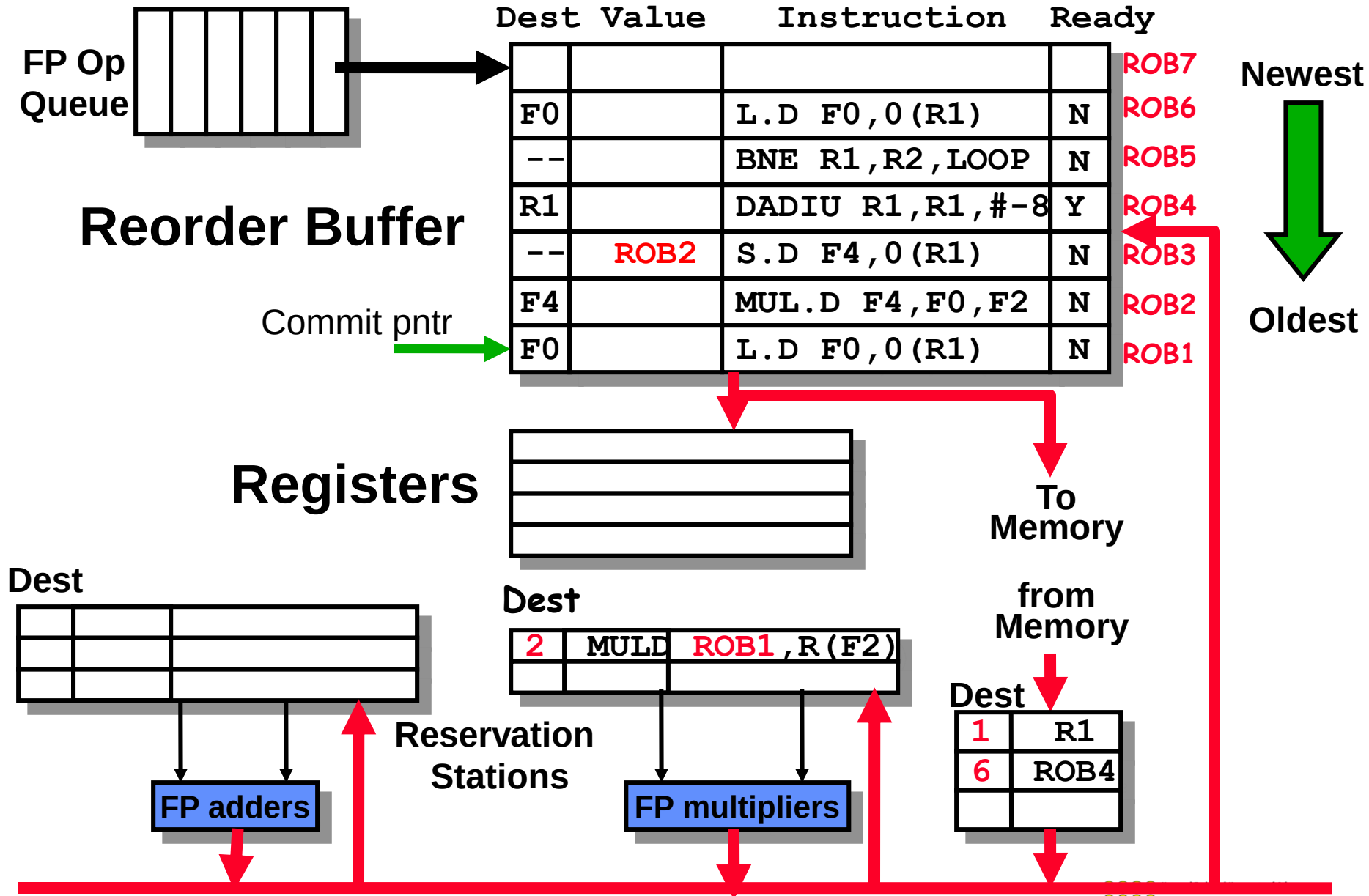
Tomasulo With Reorder buffer(4)



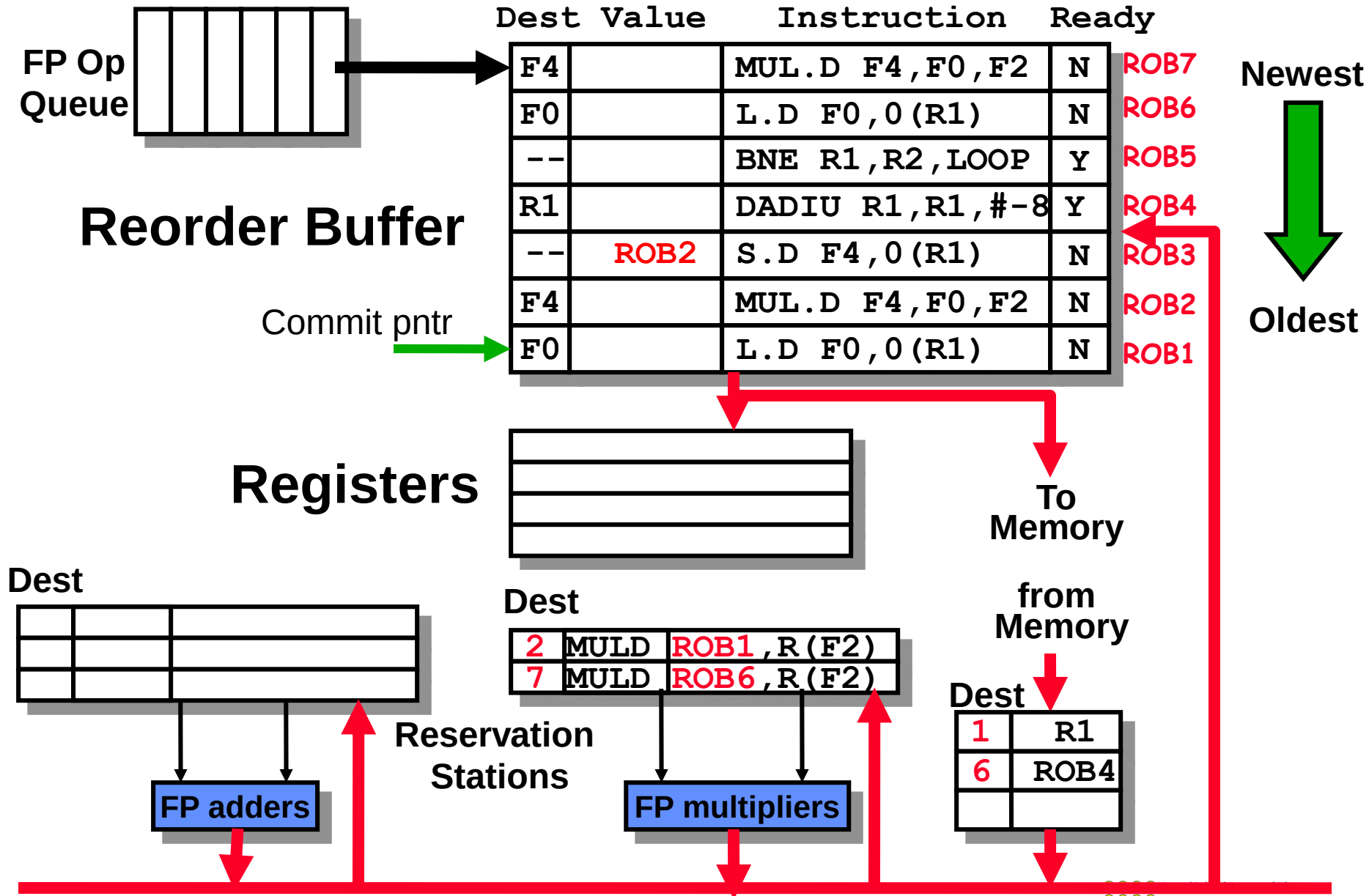
Tomasulo With Reorder buffer(5)



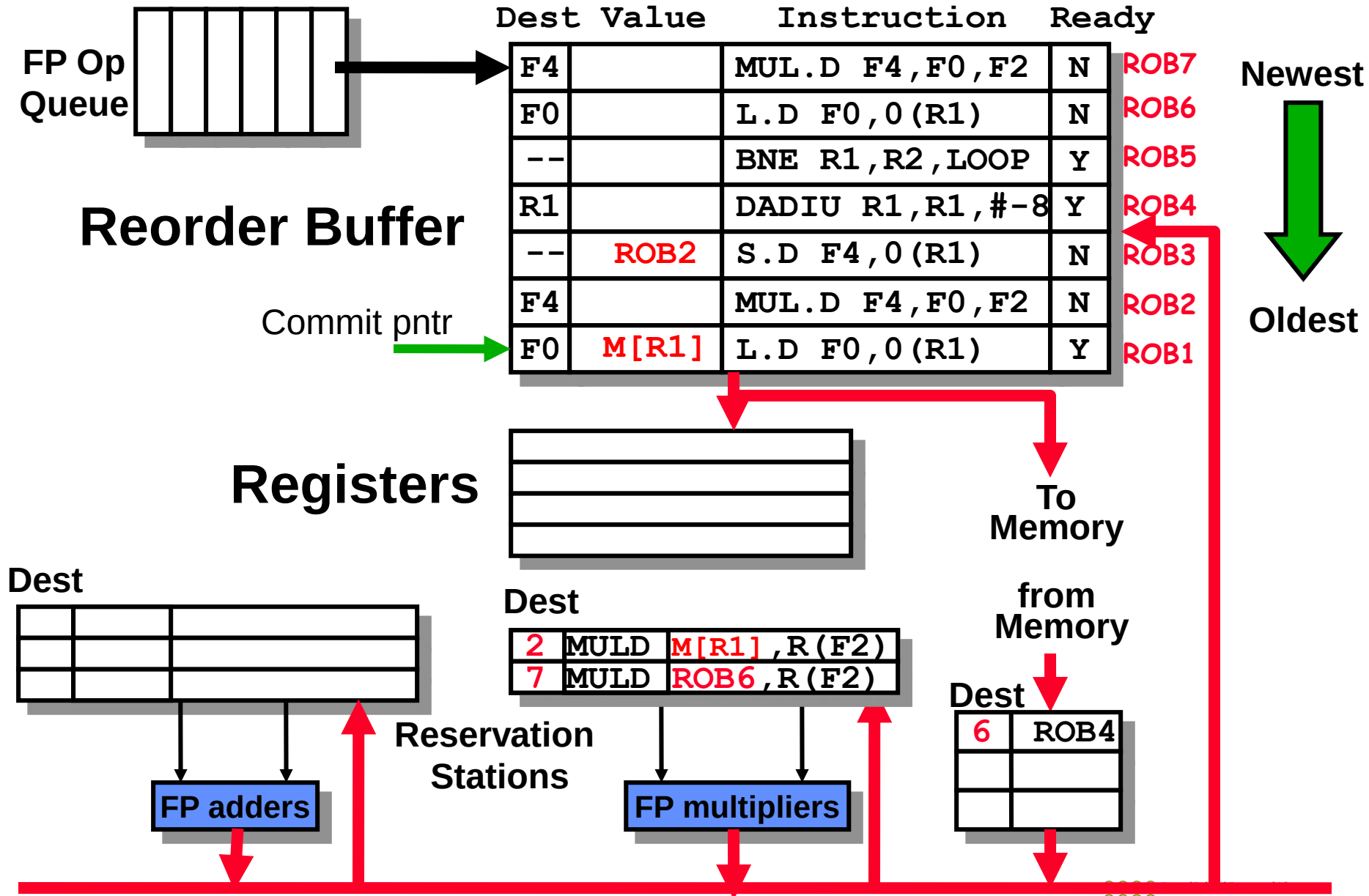
Tomasulo With Reorder buffer(6)



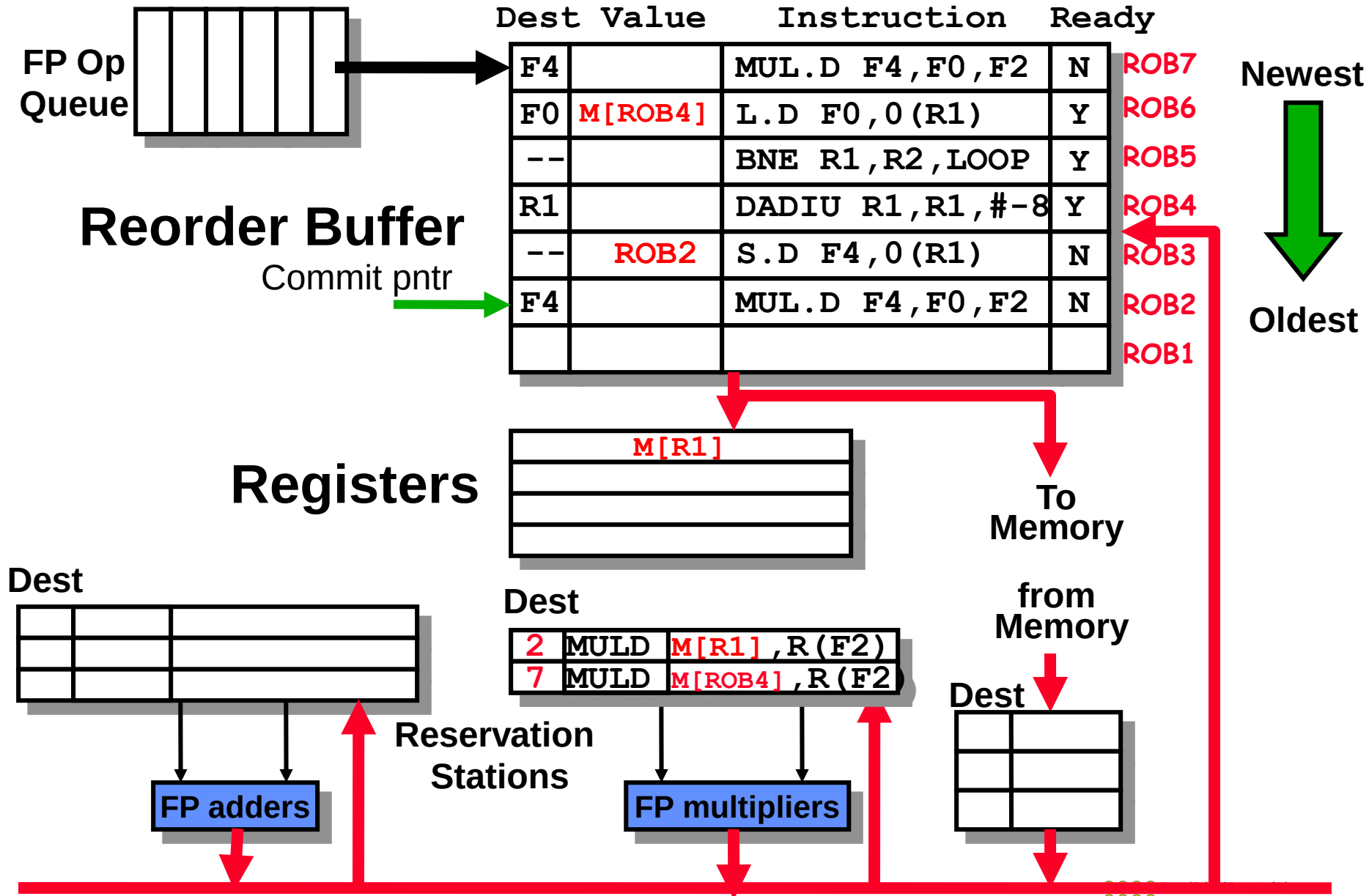
Tomasulo With Reorder buffer(7)



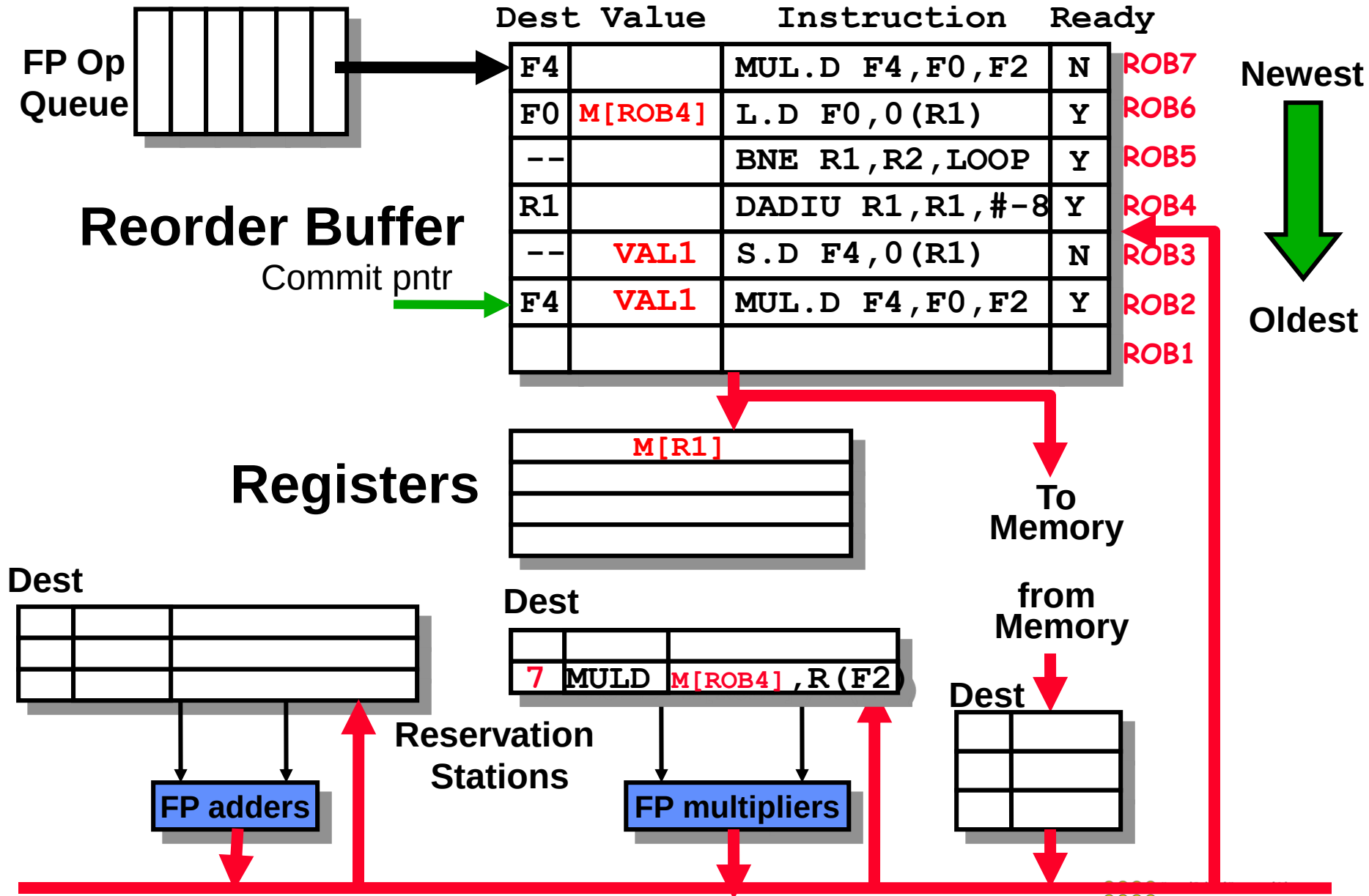
Tomasulo With Reorder buffer(8)



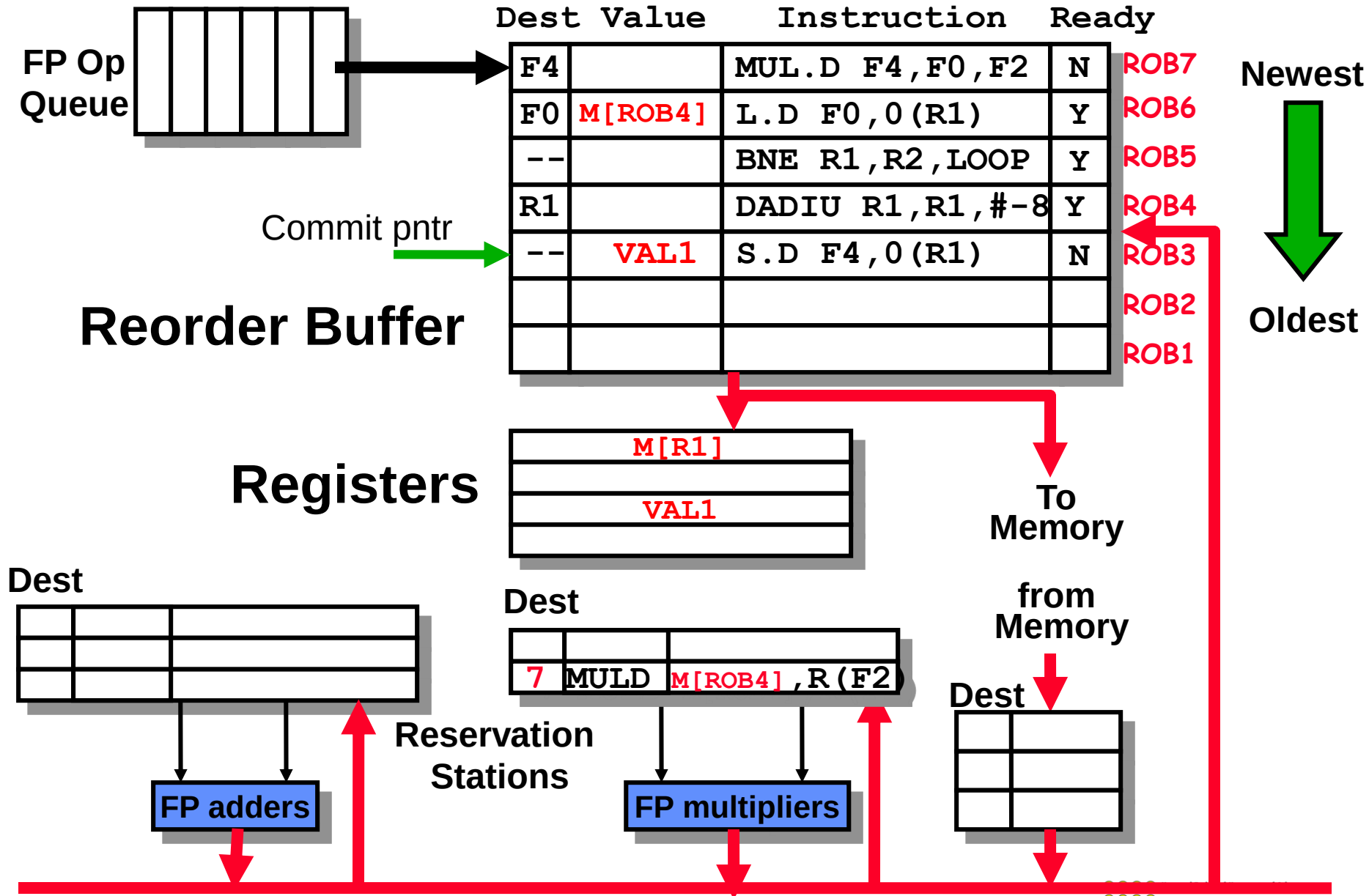
Tomasulo With Reorder buffer(9)



Tomasulo With Reorder buffer(12)



Tomasulo With Reorder buffer(13)



Tomasulo With Reorder buffer(13)

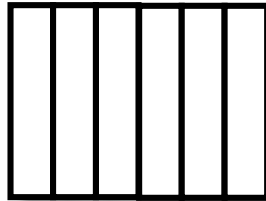
- Στο σημείο αυτό γίνεται commit το SD, ενώ καθώς αδειάζει ο ROB εισέρχονται καινούριες εντολές.
- Οι άλλες εντολές (DADDIU,BNE,LD) ενώ έχουν εκτελεστεί **δεν έχουν επικυρωθεί ακόμα (commit).**
- Αν η πρόβλεψη **taken** για το BNE ήταν **σωστή**, οι επόμενες εντολές που έχουν εκτελεστεί επικυρώνονται κανονικά
- Αν ανακαλύψουμε όμως ότι ήταν **λάθος**, τότε απλά διαγράφουμε τις εντολές από το BNE και πάνω μέσα στον ROB (roll-back). Η εκτέλεση συνεχίζεται από το σωστό σημείο.
 - Aggressive συστήματα θα μπορούσαν να κάνουν τη διαγραφή και πριν ο commit_ptr φτάσει στο BNE

Exceptions and Interrupts

- IBM 360/91 invented “*imprecise* interrupts”
 - Computer stopped at this PC; its likely close to this address
 - Not so popular with programmers
 - Also, what about Virtual Memory? (Not in IBM 360!)
 - » Need to restart the faulting instruction
- Technique for both precise interrupts/exceptions and speculation: in-order commit
 - If we speculate and are wrong, need to back up and restart execution to point at which we predicted incorrectly
 - This is exactly same as need to do with precise exceptions
 - Basic mechanism: roll-back
- Exceptions are handled by not recognizing the exception until instruction that caused it is ready to commit in ROB
 - If a speculated instruction raises an exception, the exception is recorded in the ROB
 - This is why we have reorder buffers in *all* new processors

Exceptions With Reorder Buffer:

FP Op Queue



Reorder Buffer

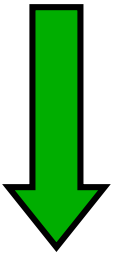
A: DIVD has exception
Mark in ROB entry

RdValue Instr Done?

--	M[10]	ST 0 (R3) , F4	Y
F0	<val2>	ADDD F0 , F4 , F6	Y
F4	M[10]	LD F4 , 0 (R3)	Y
--		BNE F2 , <...>	N
F2	NaN	DIVD F2 , F10 , F6	Ex

ROB7
ROB6
ROB5
ROB4
ROB3
ROB2
ROB1

Newest



Oldest

RF

F10: val3
F0: M[30]

Dest

Reservation Stations

Dest

FP adder

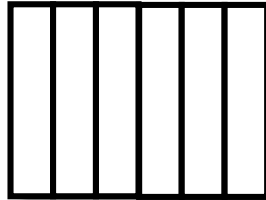
FP multiplier

To
Memory
from
Memory

Dest

Exceptions With Reorder Buffer:

FP Op Queue



Reorder Buffer

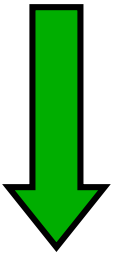
A: DIVD commits
 → Exception is taken!!!
 LD, ADDD, ST killed
 Need: Status, PC

RdValue Instr Done?

--	M[10]	ST 0(R3), F4	Y
F0	<val2>	ADDD F0, F4, F6	Y
F4	M[10]	LD F4, 0(R3)	Y
--		BNE F2, <...>	N
F2	NaN	DIVD F2, F10, F6	Ex

ROB7
ROB6
ROB5
ROB4
ROB3
ROB2
ROB1

Newest



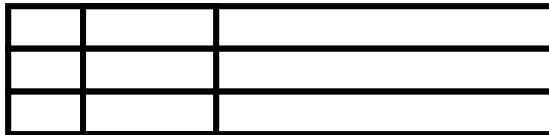
Oldest

ARF

F10: val3
F0: M[30]

To
Memory
from
Memory

Dest



FP adder

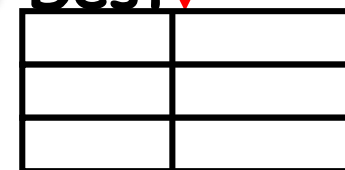
Dest



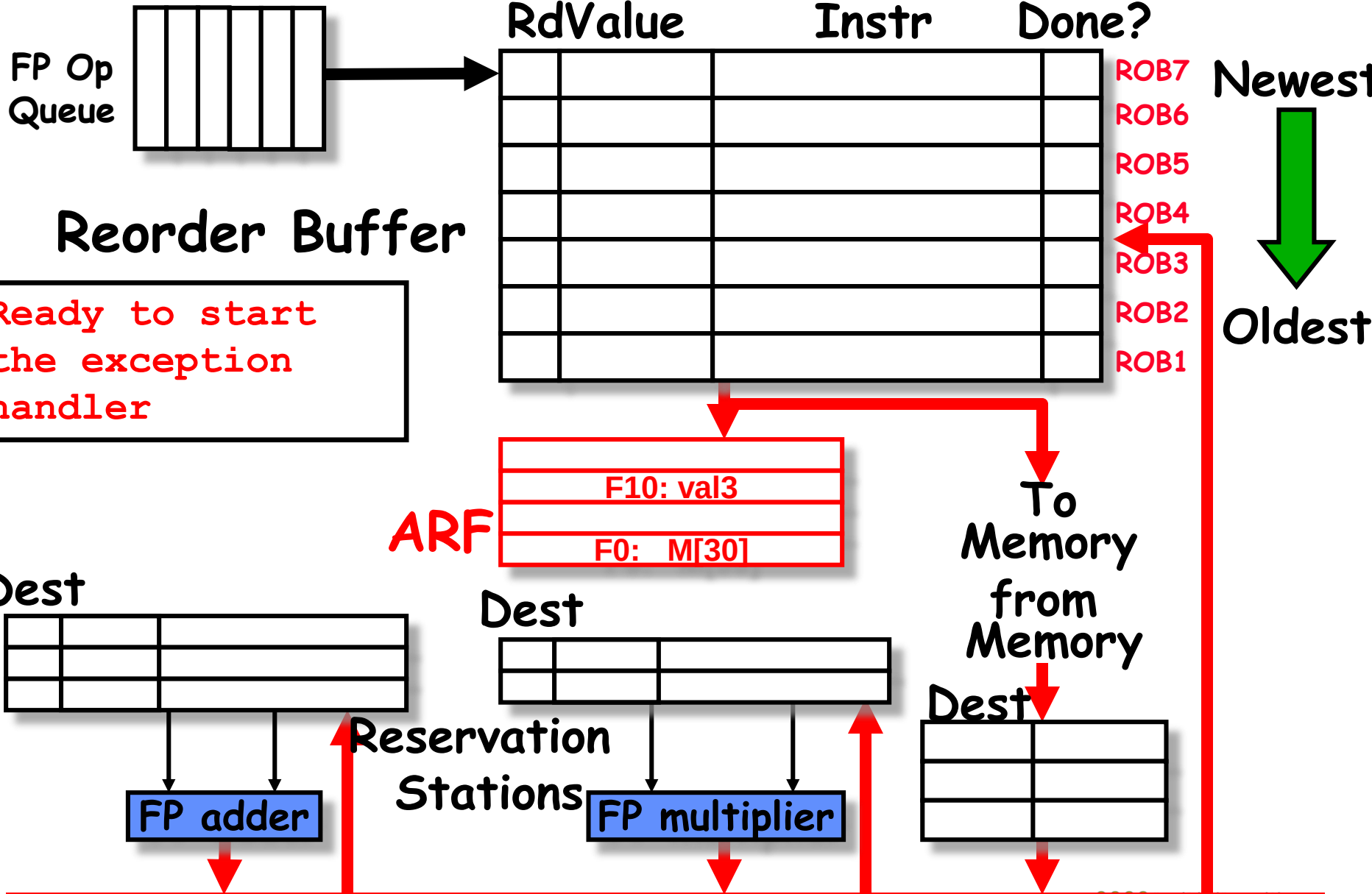
FP multiplier

Reservation
Stations

Dest

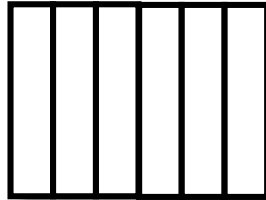


Exceptions With Reorder Buffer:



Tomasulo With Reorder Buffer:

FP Op Queue



Reorder Buffer

B: DIVD completes successfully

RdValue Instr Done?

--	M[10]	ST 0 (R3) , F4	Y
F0	<val2>	ADDD F0 , F4 , F6	Y
F4	M[10]	LD F4 , 0 (R3)	Y
--		BNE F2 , <...>	N
F2	<val4>	DIVD F2 , F10 , F6	Y

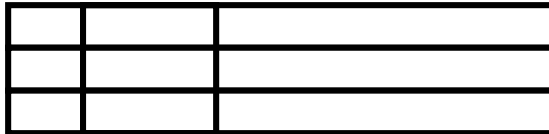
ROB7
ROB6
ROB5
ROB4
ROB3
ROB2
ROB1

Newest
↓
Oldest

RF

F10: val3
F2: val4
F0: M[30]

Dest



FP adder

Dest

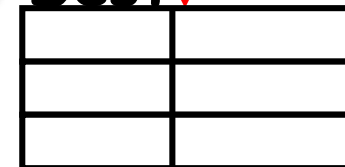


FP multiplier

Reservation Stations

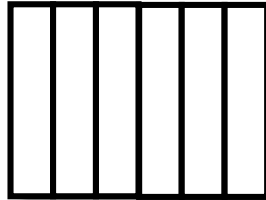
To Memory
from Memory

Dest



Tomasulo With Reorder Buffer:

FP Op Queue



Reorder Buffer

B: DIVD commits
RF[F2] = val4

RdValue Instr Done?

--	M[10]	ST 0 (R3) , F4	Y
F0	<val2>	ADDD F0 , F4 , F6	Y
F4	M[10]	LD F4 , 0 (R3)	Y
--		BNE F2 , <...>	N

ROB7
ROB6
ROB5
ROB4
ROB3
ROB2
ROB1

Newest
↓
Oldest

RF

F10: val3
F2: val4
F0: M[30]

Dest

Reservation Stations

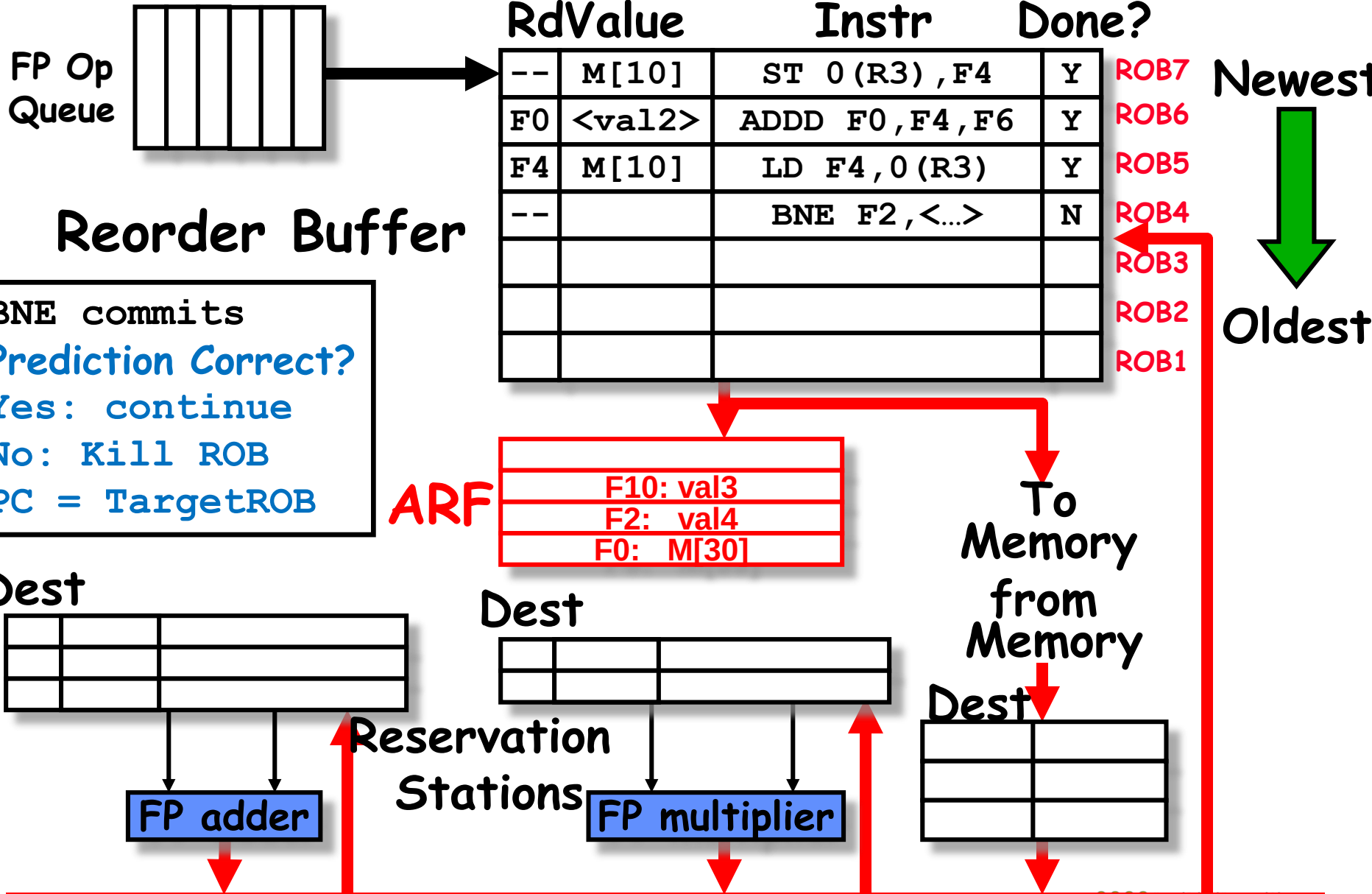
FP adder

FP multiplier

To Memory
from Memory

Dest

Tomasulo With Reorder Buffer:



Precise State with ROB

- ROB maintains precise state and allows speculation
 - Waits until precise condition reaches retire/commit stage
 - (Or until branch is noted mis-predicted)
 - Clear ROB, RS, and register status table (Flush)
 - Service exception and then Restart from True Branch target
- Need to do similar things with memory ops
 - Called Memory Ordering Buffer (MOB)
 - Completed stores are written to MOB then complete (write to memory) in-order (when they reach head of buffer)

Precise Points

Commit == free resources!

- ISA: each instruction defines a “clean” point to stop
- BUT, why do we need to stop/restart?
 - External Interrupts: we can choose *if* and *when* to stop
 - » Maskable interrupts, priorities, etc.
 - Exceptions: Internal to the program, cannot choose if and when!
 - » Non-restartable: fatal, kill program, all is lost, easy!
 - » Restartable: Which ones? Not all instructions!!!
 - Wrong prediction (branch, etc.)
- Fine, we have to restart the execution, but where?
 - What if we restart from an *earlier point* and re-execute more instructions?
Is it valid? Is the cost acceptable?
- Commit points:
 - Branch, Ld/St
- Perhaps branch only? (remember basic blocks?)

Precise Points

Needed restart points in the following program?

Label:

LD	F6	34+	R2	✓ address/perm
MULTD	F0	F2	F4	✓ overflow
LD	F2	45+	R3	✓ address/prem
SUBD	F8	F6	F2	✓ overflow
subi	R2	R2	1	
bne	R2	R8	Label	✓ misprediction

Assume exception in second LD. Can I re-execute *all* instructions (from Label)?

- Yes, if I have the ARF+PC+Mem for Label

Coarse-grain Precise Points

- Keep enough info only for *some* points in the program
Branch (and perhaps mem) instructions
- On “problem” (exception/misprediction/etc)
 - Return to the latest safe point before problem i.e. undo/kill all instructions after that point, restart from PC
 - All memory effects should be undoable:
Writes should not reach memory until committed (i.e. keep results in write buffers!)

Memory Hazards

- **WAW** hazards: two stores on the same location
 - Check if `store1_address == store2_address`, τότε υπάρχει WAW
 - What do I need to do?
 - Delay store2 until store1 has committed
 - Store write value is buffered in the store queue, in-order writes => No WAW hazard.
 - **WAR** hazards: load followed by a store on the same location
 - Check if `load_address == store_address`, τότε υπάρχει WAR
 - What do I need to do?
 - Delay the store until load has read its value.
 - Store write value is buffered in the store queue, in-order writes => No WAR hazard.
- => No **WAW**, **WAR** hazards

Memory Hazards

- Για την αποφυγή **RAW** hazards ακολουθούμε τα εξής βήματα:
 - Αν κάποιο store προηγείται ενός load, καθυστερούμε το load μέχρι το store να υπολογίσει το address του
 - Αν $\text{load_address} = \text{store_address}$, τότε υπάρχει RAW και:
 - Αν η τιμή του store **είναι** γνωστή, την περνάμε στο load
 - Αν η τιμή του store **δεν είναι** γνωστή, τότε το load χρησιμοποιεί ως source το **νούμερο του ROB** που περιέχει το store (store-to-load forwarding). Ο κώδικας είναι πλέον πιο γρήγορος από τον αρχικό!
 - Αποστολή του request στη μνήμη

Memory Disambiguation: Sorting out RAW Hazards in memory

- Question: Given a load that follows a store in program order, are the two related?
 - (Alternatively: is there a RAW hazard between the store and the load)?

Eg: st 0 (R2) , R5
 ld R6 , 0 (R3)

- Can we go ahead and start the load early?
 - Store address could be delayed for a long time by some calculation that leads to R2 (divide?).
 - We might want to issue/begin execution of both operations in same cycle.
 - Today: Answer is that we are not allowed to start load until we know that address $0(R2) \neq 0(R3)$
 - Later: We might guess at whether or not they are dependent (called “dependence speculation”) and use reorder buffer to fixup if we are wrong.

Hardware Support for Memory Disambiguation

- Need buffer to keep track of all outstanding stores to memory, in program order.
 - Keep track of address (when becomes available) and value (when becomes available)
 - FIFO ordering: will retire stores from this buffer in program order
- When issuing a load, record current head of store queue (know which stores are ahead of you).
- When have address for load, check store queue:
 - If *any* store prior to load is waiting for its address, stall load.
 - If load address matches earlier store address (associative lookup), then we have a *memory-induced RAW hazard*:
 - » store value available $\Rightarrow$ return value
 - » store value not available $\Rightarrow$ return ROB number of source
 - Otherwise, send out request to memory
- Actual stores commit in order, so no worry about WAR/WAW hazards through memory.

Multiple Issue

- To achieve $CPI < 1$, need to complete multiple instructions per clock
- Solutions:
 - Statically scheduled superscalar processors
 - VLIW (very long instruction word) processors
 - dynamically scheduled superscalar processors

Multiple Issue

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the ARM Coretex A8
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Dynamic Scheduling, Multiple Issue and Speculation

- Modern microarchitectures:
 - Dynamic scheduling + multiple issue + speculation
- Two approaches:
 - Assign reservation stations and update pipeline control table in half clock cycles
 - » Only supports 2 instructions/clock
 - Design logic to handle any possible dependencies between the instructions
 - Hybrid approaches
- Issue logic can become bottleneck

Multiple Issue

- Limit the number of instructions of a given class that can be issued in a “bundle”
 - I.e. on FP, one integer, one load, one store
- Examine all the dependencies among the instructions in the bundle
- If dependencies exist in bundle, encode them in reservation stations
- Also need multiple completion/commit

Dynamic Scheduling in Superscalar (multi issue)

- How to issue two instructions and keep in-order instruction issue for Tomasulo?
 - Assume 1 integer + 1 floating point
 - 1 Tomasulo control for integer, 1 for floating point
- Issue 2X Clock Rate, so that issue remains in order
- Only FP loads might cause dependency between integer and FP issue:
 - Replace load reservation station with a load queue; operands must be read in the order they are fetched
 - Load checks addresses in Store Queue to avoid RAW violation
 - Store checks addresses in Load Queue to avoid WAR,WAW
 - Called “[decoupled architecture](#)”

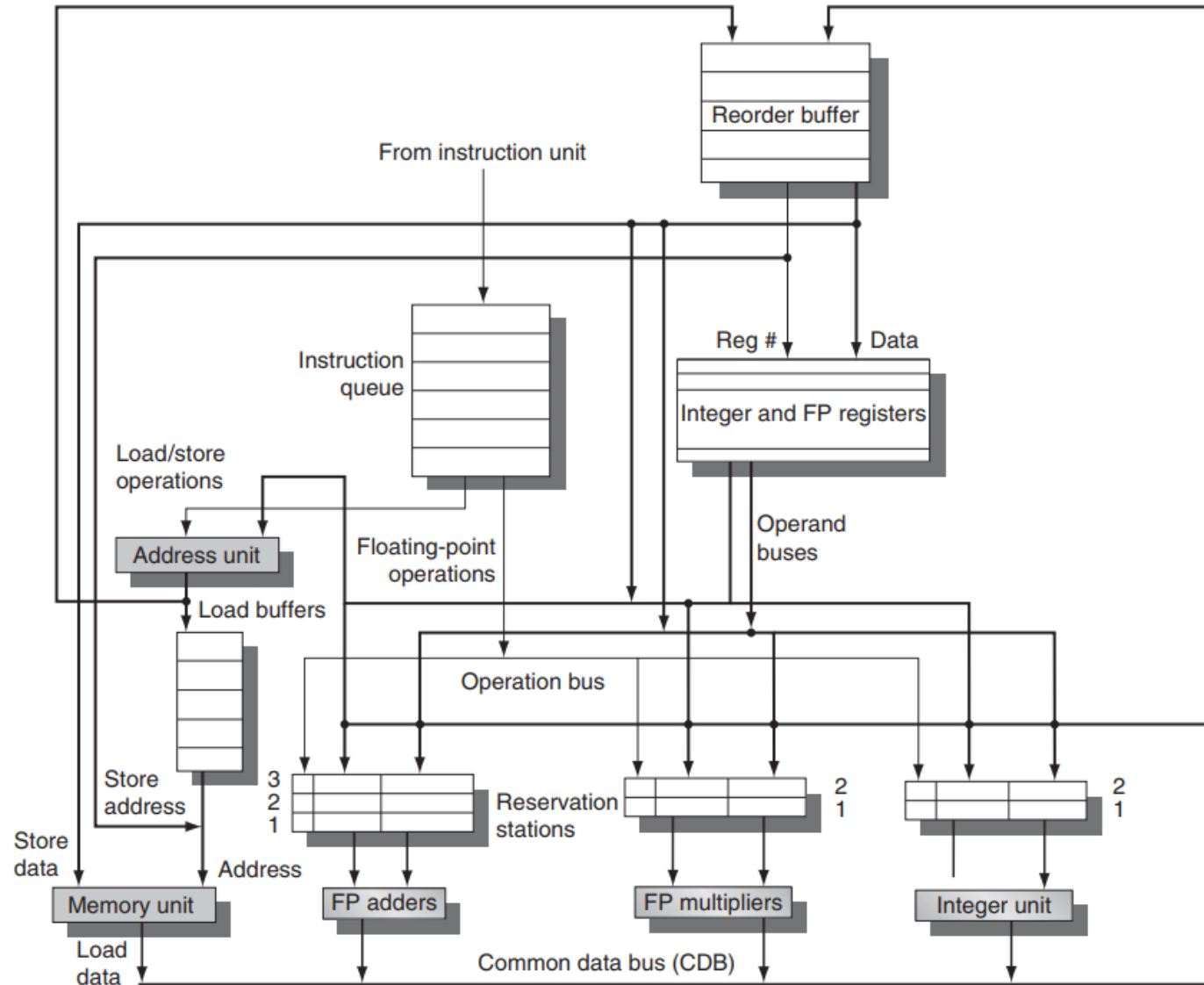
Multiple Issue Challenges

- While Integer/FP split is simple for the HW, get CPI of 0.5 only for programs with:
 - Exactly 50% FP operations
 - No hazards
- If more instructions issue at same time (in a **bundle**), **greater difficulty of decode and issue**:
 - Even 2-scalar => examine 2 opcodes, 6 register specifiers, & decide if 1 or 2 instructions can issue
 - Multiported rename logic: must be able to rename same register multiple times in one cycle!
- VLIW: tradeoff instruction space for simple decoding
 - The long instruction word has room for many operations
 - By definition, all the operations the compiler puts in the long instruction word are independent => execute in parallel
 - E.g., 2 integer operations, 2 FP ops, 2 Memory refs, 1 branch
 - » 16 to 24 bits per field => 7*16 or 112 bits to 7*24 or 168 bits wide
 - Need compiling technique that schedules across several branches

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the ARM Cortex-A8
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Figure 3.15 The five primary approaches in use for multiple-issue processors and the primary characteristics that distinguish them. This chapter has focused on the hardware-intensive techniques, which are all some form of superscalar. Appendix H focuses on compiler-based approaches. The EPIC approach, as embodied in the IA-64 architecture, extends many of the concepts of the early VLIW approaches, providing a blend of static and dynamic approaches.

Multi issue with speculation



Loop example in a Dual Issue processor

```
Loop:   LD      R2,0(R1)      ;R2=array element
        DADDIU  R2,R2,#1      ;increment R2
        SD      R2,0(R1)      ;store result
        DADDIU  R1,R1,#8      ;increment pointer
        BNE     R2,R3,LOOP     ;branch if not last element
```

Dual Issue without Speculation

Iteration number	Instructions		Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	LD	R2,0(R1)	1	2	3	4	First issue
1	DADDIU	R2,R2,#1	1	5		6	Wait for LW
1	SD	R2,0(R1)	2	3	7		Wait for DADDIU
1	DADDIU	R1,R1,#8	2	3		4	Execute directly
1	BNE	R2,R3,LOOP	3	7			Wait for DADDIU
2	LD	R2,0(R1)	4	8	9	10	Wait for BNE
2	DADDIU	R2,R2,#1	4	11		12	Wait for LW
2	SD	R2,0(R1)	5	9	13		Wait for DADDIU
2	DADDIU	R1,R1,#8	5	8		9	Wait for BNE
2	BNE	R2,R3,LOOP	6	13			Wait for DADDIU
3	LD	R2,0(R1)	7	14	15	16	Wait for BNE
3	DADDIU	R2,R2,#1	7	17		18	Wait for LW
3	SD	R2,0(R1)	8	15	19		Wait for DADDIU
3	DADDIU	R1,R1,#8	8	14		15	Wait for BNE
3	BNE	R2,R3,LOOP	9	19			Wait for DADDIU

Loop:

LD	R2,0(R1)	;R2=array element
DADDIU	R2,R2,#1	;increment R2
SD	R2,0(R1)	;store result
DADDIU	R1,R1,#8	;increment pointer
BNE	R2,R3,LOOP	;branch if not last element

Dual Issue with Speculation

Iteration number	Instructions		Issues at clock number	Executes at clock number	Read access at clock number	Write CDB at clock number	Commits at clock number	Comment
1	LD	R2,0(R1)	1	2	3	4	5	First issue
1	DADDIU	R2,R2,#1	1	5		6	7	Wait for LW
1	SD	R2,0(R1)	2	3			7	Wait for DADDIU
1	DADDIU	R1,R1,#8	2	3		4	8	Commit in order
1	BNE	R2,R3,LOOP	3	7			8	Wait for DADDIU
2	LD	R2,0(R1)	4	5	6	7	9	No execute delay
2	DADDIU	R2,R2,#1	4	8		9	10	Wait for LW
2	SD	R2,0(R1)	5	6			10	Wait for DADDIU
2	DADDIU	R1,R1,#8	5	6		7	11	Commit in order
2	BNE	R2,R3,LOOP	6	10			11	Wait for DADDIU
3	LD	R2,0(R1)	7	8	9	10	12	Earliest possible
3	DADDIU	R2,R2,#1	7	11		12	13	Wait for LW
3	SD	R2,0(R1)	8	9			13	Wait for DADDIU
3	DADDIU	R1,R1,#8	8	9		10	14	Executes earlier
3	BNE	R2,R3,LOOP	9	13			14	Wait for DADDIU

Loop: LD R2,0(R1) ;R2=array element
 DADDIU R2,R2,#1 ;increment R2
 SD R2,0(R1) ;store result
 DADDIU R1,R1,#8 ;increment pointer
 BNE R2,R3,LOOP ;branch if not last element

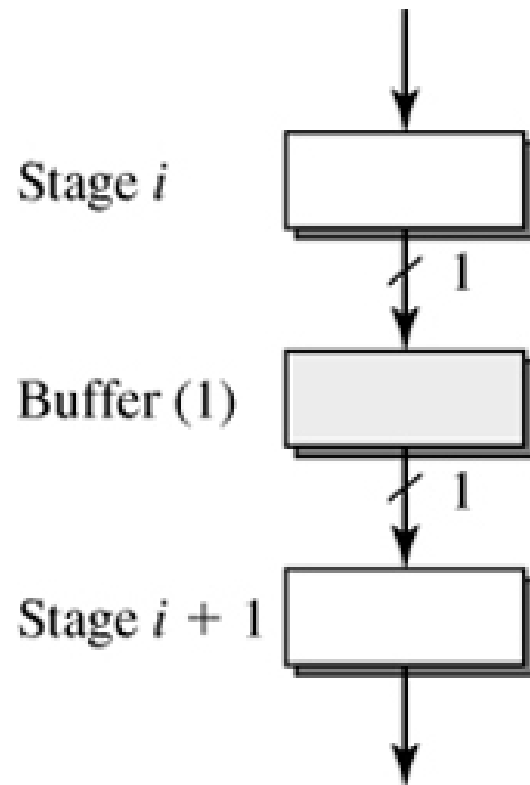
slides from Lipasti/Shen

Out-of-order εκτέλεση:

επανεξέταση υπερβαθμωτών αρχιτεκτονικών...

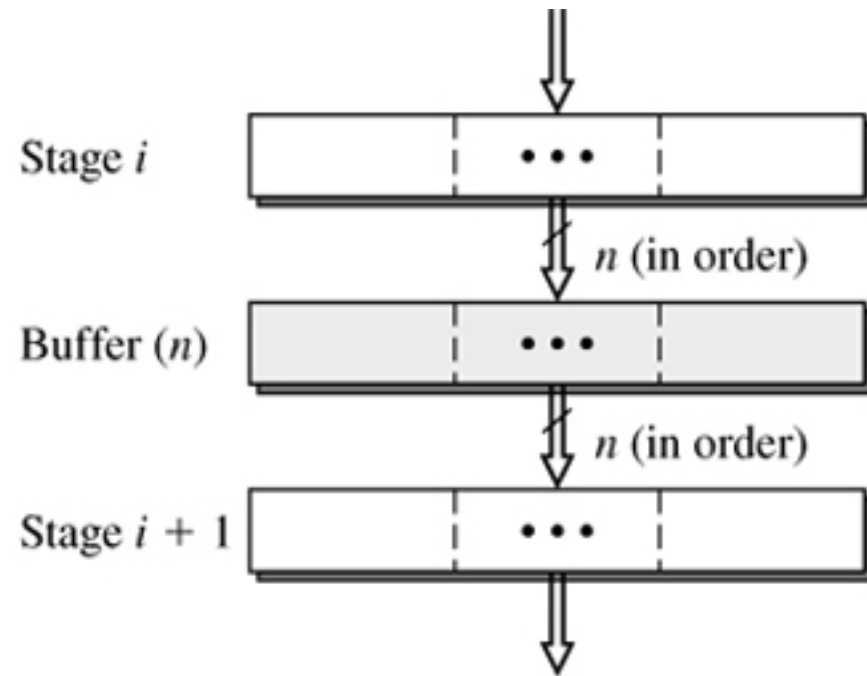
Βαθμωτή αρχιτεκτονική αγωγού

- μεταξύ δύο σταδίων του pipeline υπάρχει πάντα ένας ενδιάμεσος καταχωρητής δεδομένων (pipeline register ή buffer)
- στις βαθμωτές σωληνώσεις, κάθε buffer έχει μία μόνο θύρα εισόδου και μία εξόδου



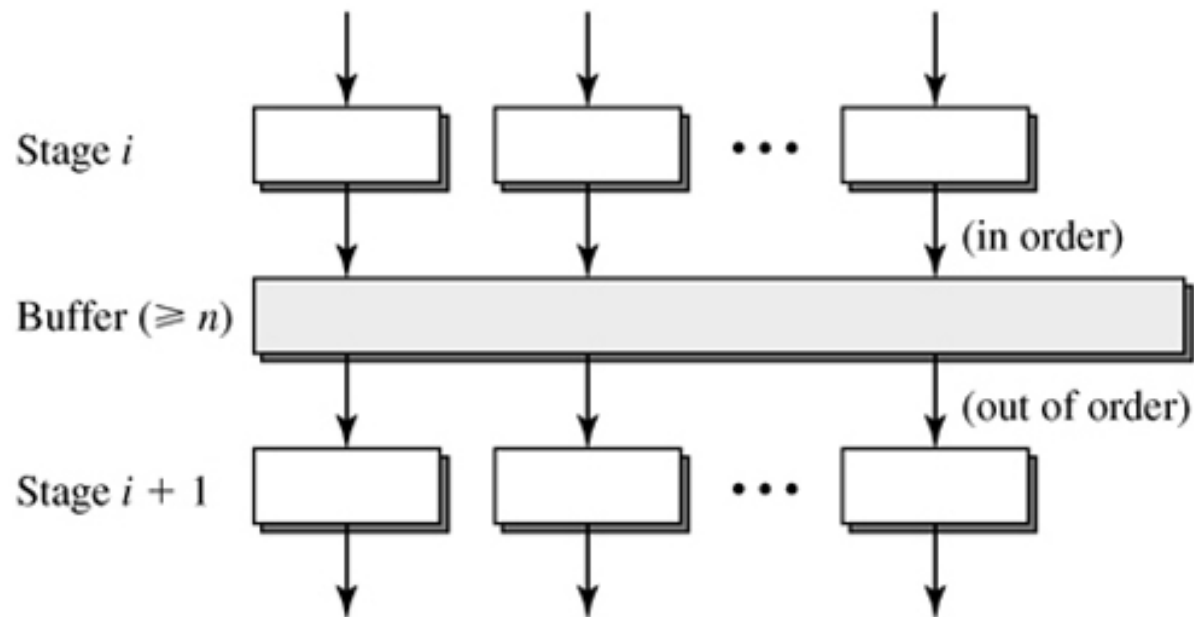
Υπερβαθμωτή αρχιτεκτονική αγωγού

- κάθε buffer έχει πολλαπλές θύρες εισόδου και εξόδου, και αποτελείται από πολλαπλούς ενδιάμεσους καταχωρητές
- κάθε ενδιάμεσος καταχωρητής μπορεί να λάβει δεδομένα μόνο από μία θύρα
- δεν υπάρχει δυνατότητα διακίνησης των δεδομένων μεταξύ των καταχωρητών του ίδιου buffer



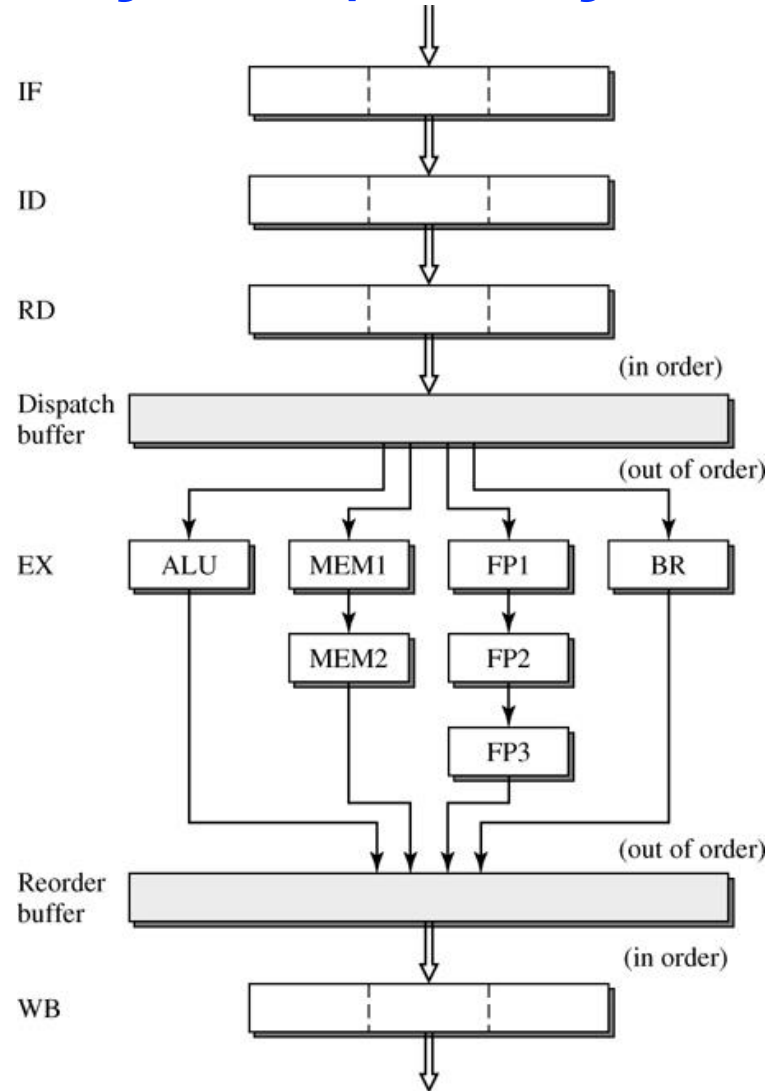
Υπερβαθμωτή αρχιτεκτονική αγωγού – εκτέλεση εκτός σειράς (out-of-order)

- οι εντολές που καθυστερούν μέσα στη σωλήνωση (stall) παρακάμπτονται και συνεχίζεται η ροή εκτέλεσης με τις επόμενες εντολές
- out-of-order execution ή dynamic scheduling



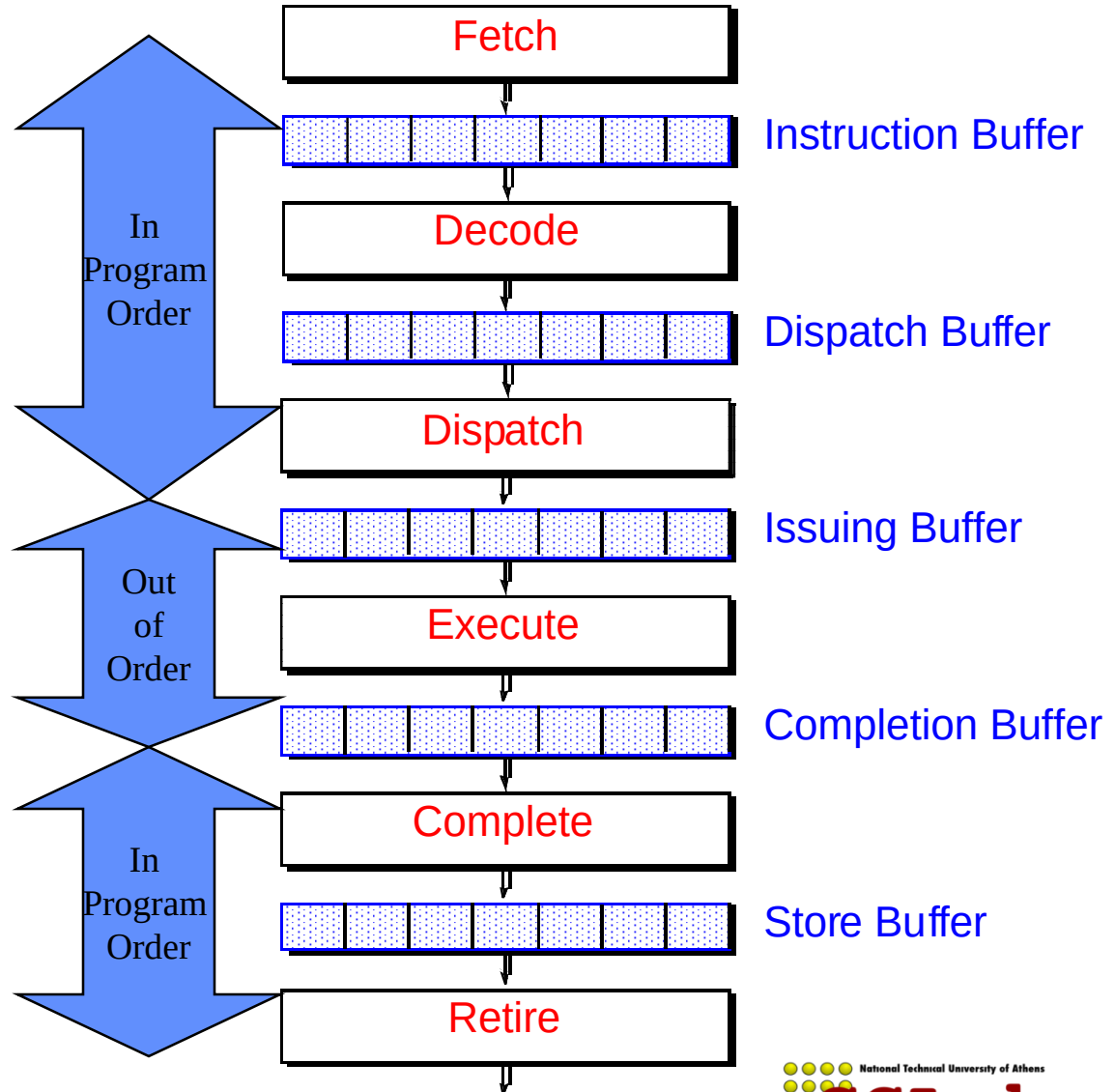
Δυναμική αρχιτεκτονική αγωγού, με πολλαπλές, ετερογενείς σωληνώσεις

- ο dispatch buffer λαμβάνει τις εντολές εν σειρά, και τις κατανέμει στις λειτουργικές μονάδες εκτός σειράς
- όταν οι εντολές ολοκληρώσουν την εκτέλεσή τους, ο reorder buffer τις αναδιατάσσει, σύμφωνα με τη σειρά που υπαγορεύει το πρόγραμμα, προκειμένου να πιστοποιήσει τη σωστή ολοκλήρωσή τους



Γενική μορφή υπερβαθμωτής αρχιτεκτονικής αγωγού

- το στάδιο εκτέλεσης (execute) μπορεί να περιλαμβάνει πολλαπλές διαφορετικού τύπου σωληνώσεις, με διαφορετικό latency η κάθε μία
- αναγκαία τα στάδια dispatch και complete, για την αναδιάταξη και επαναφορά των εντολών σε σειρά
- παρεμβολή ενδιάμεσων buffers ανάμεσα σε διαδοχικά στάδια

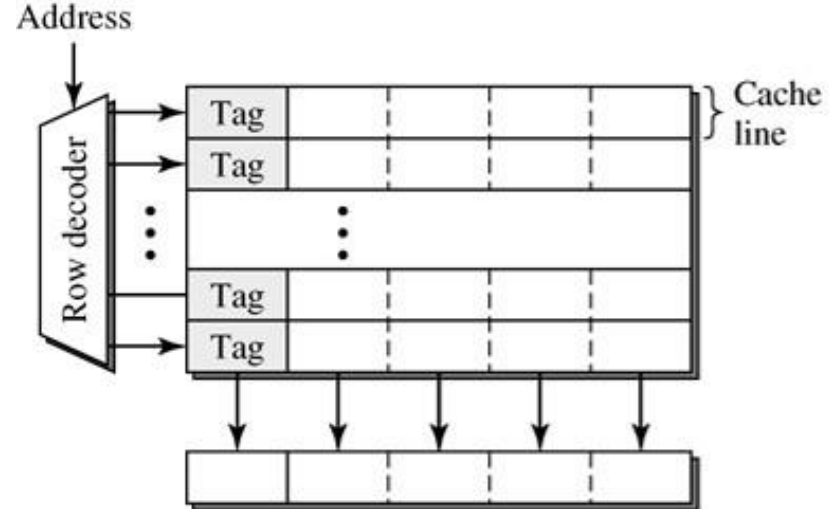


Superscalar Pipeline Design

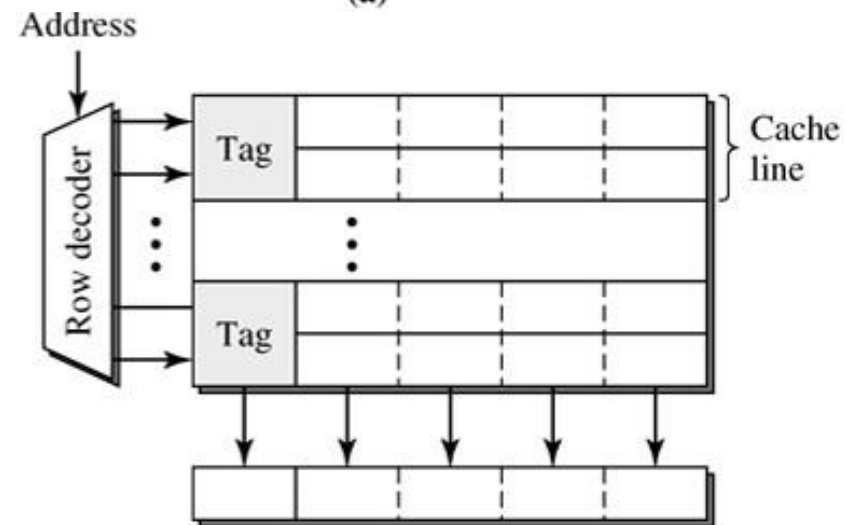
- Instruction Fetching Issues
- Instruction Decoding Issues
- Instruction Dispatching Issues
- Instruction Execution Issues
- Instruction Completion & Retiring Issues

Ανάγνωση εντολής (Instruction Fetch)

- Σε μία αρχιτεκτονική πλάτους s , πρέπει διαβάζονται s εντολές σε κάθε κύκλο μηχανής από την κρυφή μνήμη εντολών (I-cache)
- Σε κάθε προσπέλαση της I-cache διαβάζεται μία ολόκληρη γραμμή (s εντολές ανά γραμμή): 1 cache line ανά γραμμή της I-cache (a). Θα μπορούσε μία cache line να επεκτείνεται σε περισσότερες της μίας γραμμής (b).



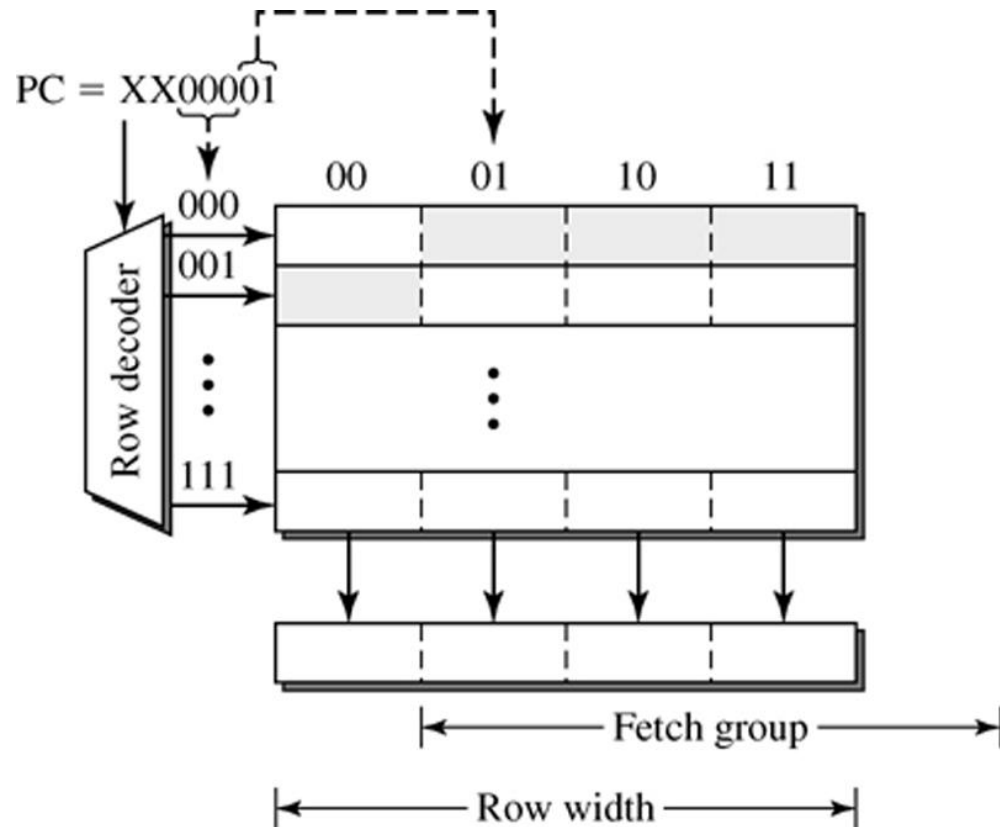
(a)



(b)

Ανάγνωση εντολής (Instruction Fetch)

- Για την ανάγνωση s εντολών σε κάθε κύκλο μηχανής θα πρέπει:
 - Οι s εντολές (που ανήκουν στο ίδιο **fetch group**) να είναι ευθυγραμμισμένες (aligned) στην I-cache



Ανάγνωση εντολής (Instruction Fetch)

- Εναλλακτικά, υπάρχουν μηχανισμοί ελέγχου της ροής των εντολών, που μπορούν να φροντίσουν για την ευθυγράμμιση των εντολών που ανήκουν στο ίδιο fetch group
 - Software (compiler static alignment)
 - Hardware (run time technique)

Αποκωδικοποίηση εντολών (Instruction Decode)

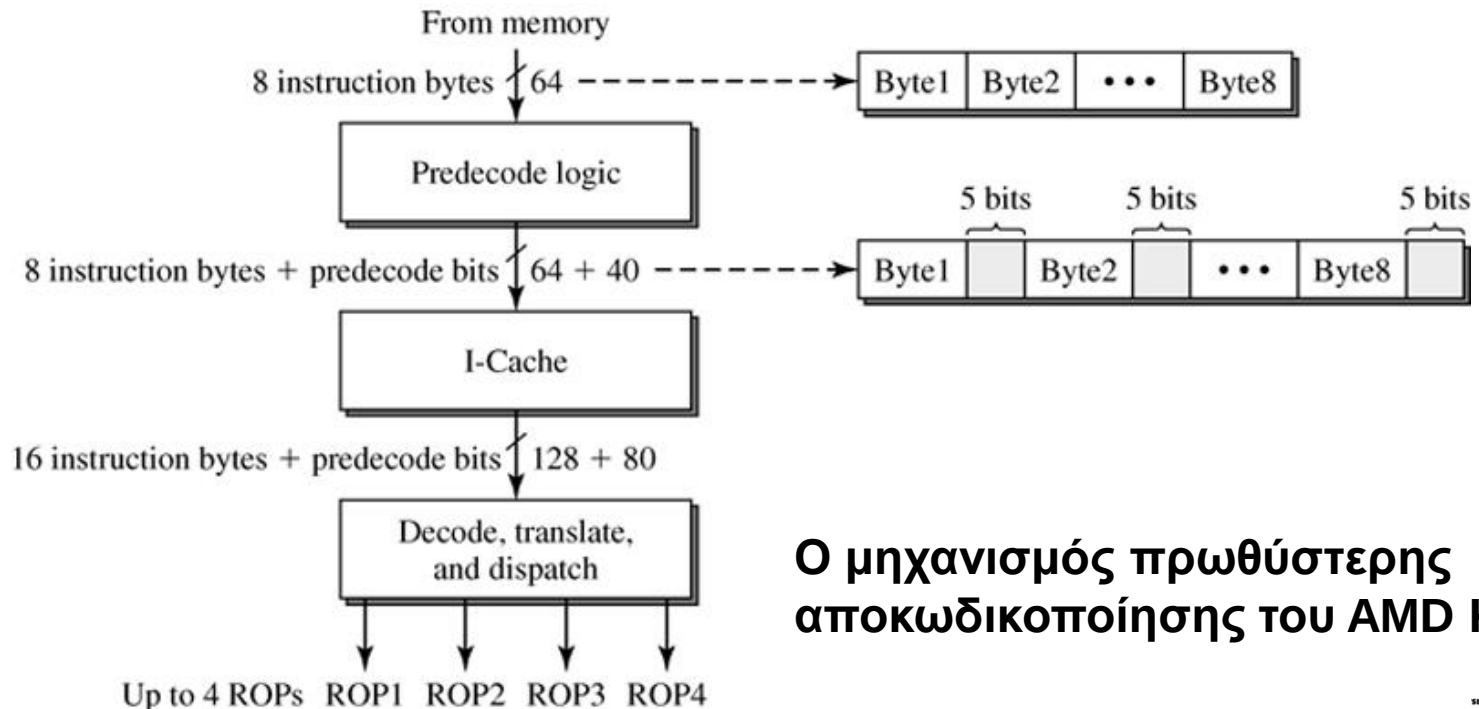
- Απλό για scalar RISC pipeline.
- Αναγνώριση των όποιων εξαρτήσεων μεταξύ εντολών – προώθηση των μη εξαρτημένων εντολών στο επόμενο στάδιο
- Αναγνώριση των εντολών άλματος
- Αποτελεί το κρισιμότερο στάδιο για την επίδοση ολόκληρης της αρχιτεκτονικής αγωγού
- Ακόμη μεγαλύτερη δυσκολία στις CISC εντολές: κάθε εντολή μεταφράζεται σε περισσότερες μικρο-εντολές (1,5-2 μops / CISC εντολή για την Intel)

Αποκωδικοποίηση εντολών (Instruction Decode)

- Η αποκωδικοποίηση των εντολών CISC σε παράλληλες αρχιτεκτονικές αγωγού ή οι πολύ πλατιές σωληνώσεις
 - Απαιτεί πολλάπλά στάδια αγωγού
 - Αυξάνει το branch penalty
- *Πώς μπορεί να αποφευχθεί η αύξηση του βάθους του τμήματος αποκωδικοποίησης στη σωλήνωση;*

Αποκωδικοποίηση εντολών (Instruction Decode)

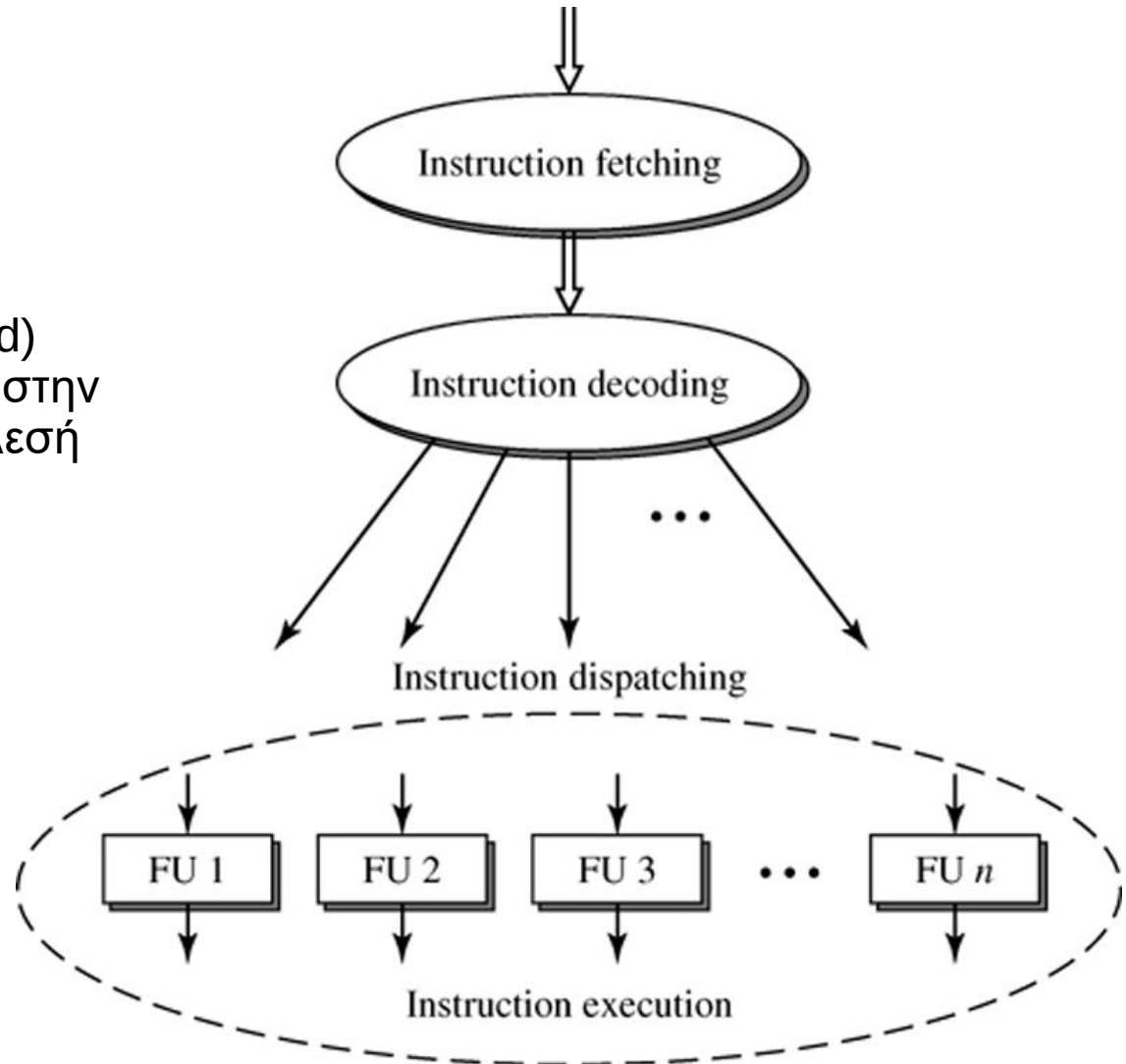
- Λύση:
 - Μερική αποκωδικοποίηση των εντολών πριν την είσοδό τους στην I-cache (**predecoding**)
- Αύξηση του I-cache miss penalty
- Αύξηση του μεγέθους της I-cache (για να συμπεριληφθούν τα predecoded bits)



Ο μηχανισμός πρωθύστερης αποκωδικοποίησης του AMD K5

Διανομή των εντολών – Instruction Dispatch

- Το στάδιο αυτό αναλαμβάνει τη μεταγωγή από την κεντρική (centralized) διαχείριση εντολών στην κατανεμημένη εκτέλεσή τους

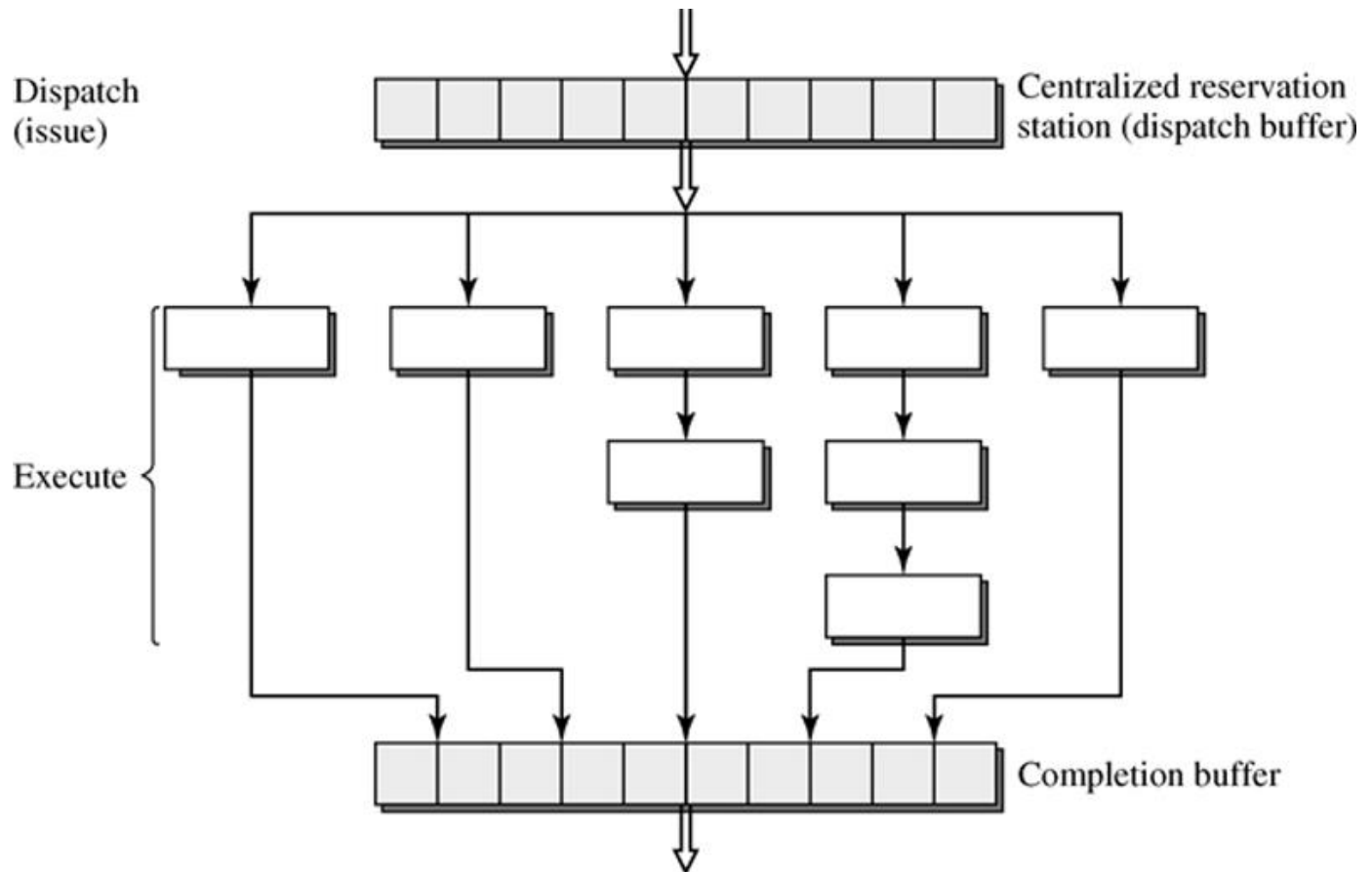


Διανομή των εντολών – Instruction Dispatch

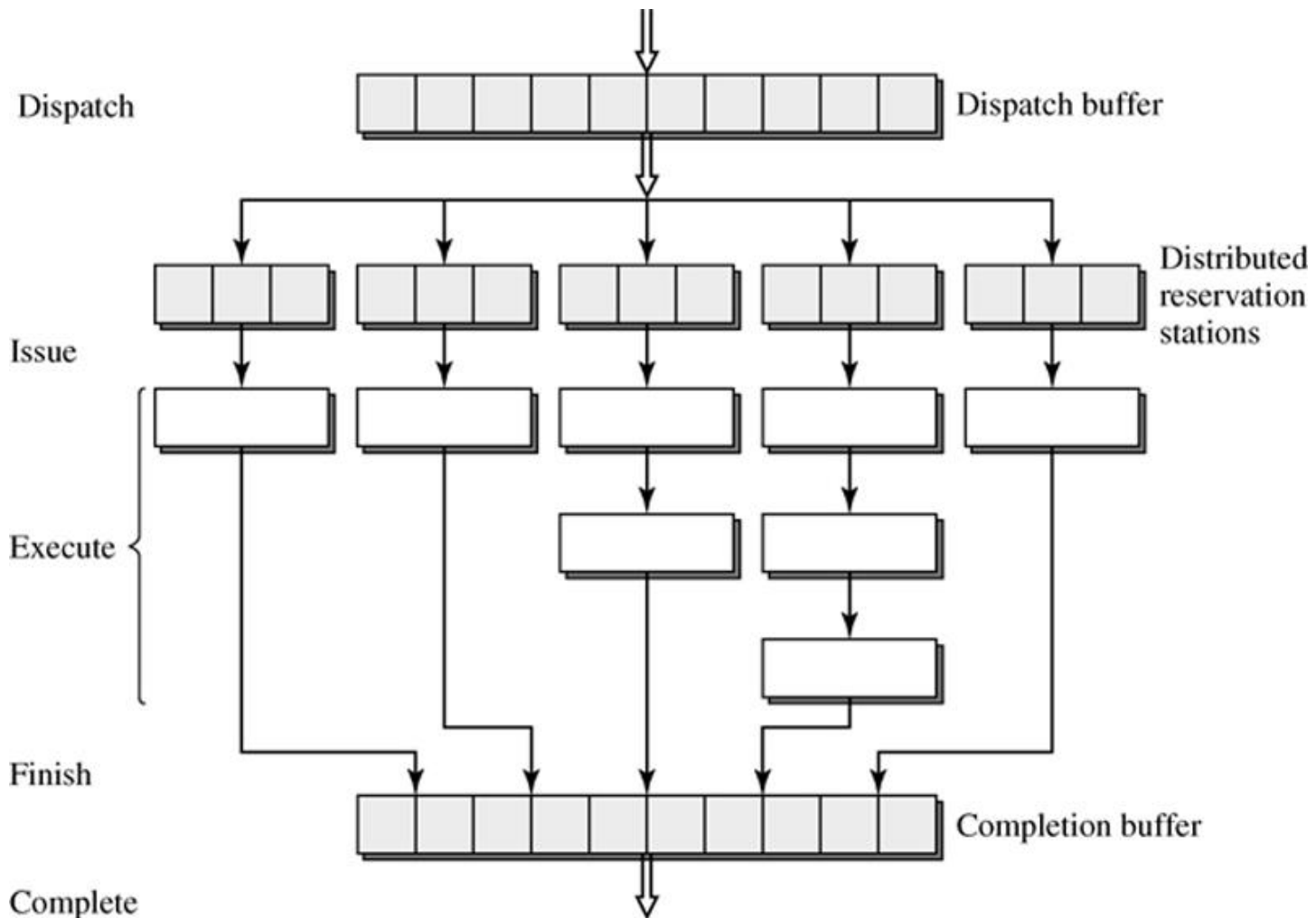
- **Reservation station:** προσωρινός καταχωρητής αποθήκευσης των αποκωδικοποιημένων εντολών που δεν έχουν διαθέσιμα όλα τα ορίσματά τους
 - Κεντρικός καταχωρητής (**centralized reservation station**)
 - » Για παράλληλες σωληνώσεις
 - Κατανεμημένοι καταχωρητές (**distributed reservation station**)
 - » Για ετερογενείς σωληνώσεις

Centralized reservation station

- Ενοποίηση των σταδίων dispatch και issue



Distributed reservation station



Εκτέλεση εντολών – Instruction Execution

- Σύγχρονες τάσεις:
 - Πολλές παράλληλες σωληνώσεις (δύσκολη η out-of-order εκτέλεση με bypassing εντολών)
 - Διαφοροποιημένες μεταξύ τους σωληνώσεις
 - Βαθιές σωληνώσεις
- Συνήθης καταμερισμός (δεν ακολουθεί τη στατιστική αναλογία των προγραμμάτων σε τύπους εντολών [40%-20%-40%]):
 - 4 μονάδες ALU
 - 1 μονάδα διακλάδωσης (μπορεί να εκτελέσει θεωρητικά (speculatively) > 1 εντολές διακλάδωσης)
 - 1 μονάδα ανάγνωσης/εγγραφής στη μνήμη (πολύ πολύπλοκη η υλοποίηση μνημών πολλαπλών εισόδων-εξόδων, μόνο με πολλαπλά banks)
 - Περισσότερες ειδικευμένες (και πιο αποδοτικές στην επίδοση) λειτουργικές μονάδες

Ολοκλήρωση και Αποδέσμευση εντολών – Instruction Completion and Retiring

- **Instruction Completion:** ενημέρωση της κατάστασης του μηχανήματος (machine state update)
- **Instruction Retiring:** ενημέρωση της μνήμης (memory state update)
- Αν η εντολή δεν περιλαμβάνει ενημέρωση της μνήμης, μετά το στάδιο ολοκλήρωσης, η εντολή αποδεσμεύεται
- Τα στάδια ολοκλήρωσης και αποδέσμευσης των εντολών πρέπει να εκτελέσουν τις εντολές εν σειρά
 - συνεπές arch. state
 - συνεπής κατάσταση μνήμης

Ολοκλήρωση και Αποδέσμευση εντολών – Instruction Completion and Retiring

- Τα στάδια εντολών:

- Fetch
- Decode
- Dispatch
- Issue
- Execute
- Finish
- Complete
- Retire

